Python_Activity_Based_Learning

Name: Badi Sujana

Roll no: 26001129

Branch: CSM

Section: 5

Selected test cases

No.	Case study number	Selected case study	Main concepts
1	13	ATM Transaction Sequence Simulation	Variables, conditions, loops, lists, tuples
2	17	Student Performance Trend Analysis	Lists, loops, conditions, calculations
3	26	Nested Loop Examination Seating Generator	Nested loops, patterns, conditions
4	42	Student Marks Matrix Analysis	Nested lists, loops, averages
5	51	Bus Passenger Occupancy Simulation	Lists, loops, conditions
6	53	Food Waste Monitoring System	Lists, percentages, conditions
7	60	Manufacturing Quality Control Matrix	Nested lists, percentages, conditions
8	71	Integrated University Fee and Eligibility Audit	Lists, tuples, conditions
9	74	Campus Festival Registration and Financial Audit	Strings, lists, loops, financial calculations
10	87	Examination Question Difficulty and Performance Analysis	Strings, nested lists, loops, percentages

Python Activity-Based Learning

Case Study 13: ATM Transaction Sequence Simulation

Test Case 1

1. Problem Statement

Develop a Python program that simulates a sequence of ATM transactions on a bank account. The program processes a series of deposits and withdrawals, maintains a simulated account balance, enforces the withdrawal rule (a withdrawal cannot exceed the available balance), rejects invalid transactions (non-positive amounts and unknown transaction types), and produces a transaction summary ending with the final balance.

2. Objectives

- To apply variables, data types and operators to maintain a simulated account balance.
- To store a transaction sequence as a list of tuples and process it using a **for** loop with tuple unpacking.
- To use conditional statements (if / elif / else, including a nested if) to validate each transaction.
- To reject invalid operations and report the reason clearly.
- To produce a transaction-by-transaction summary and the final balance.

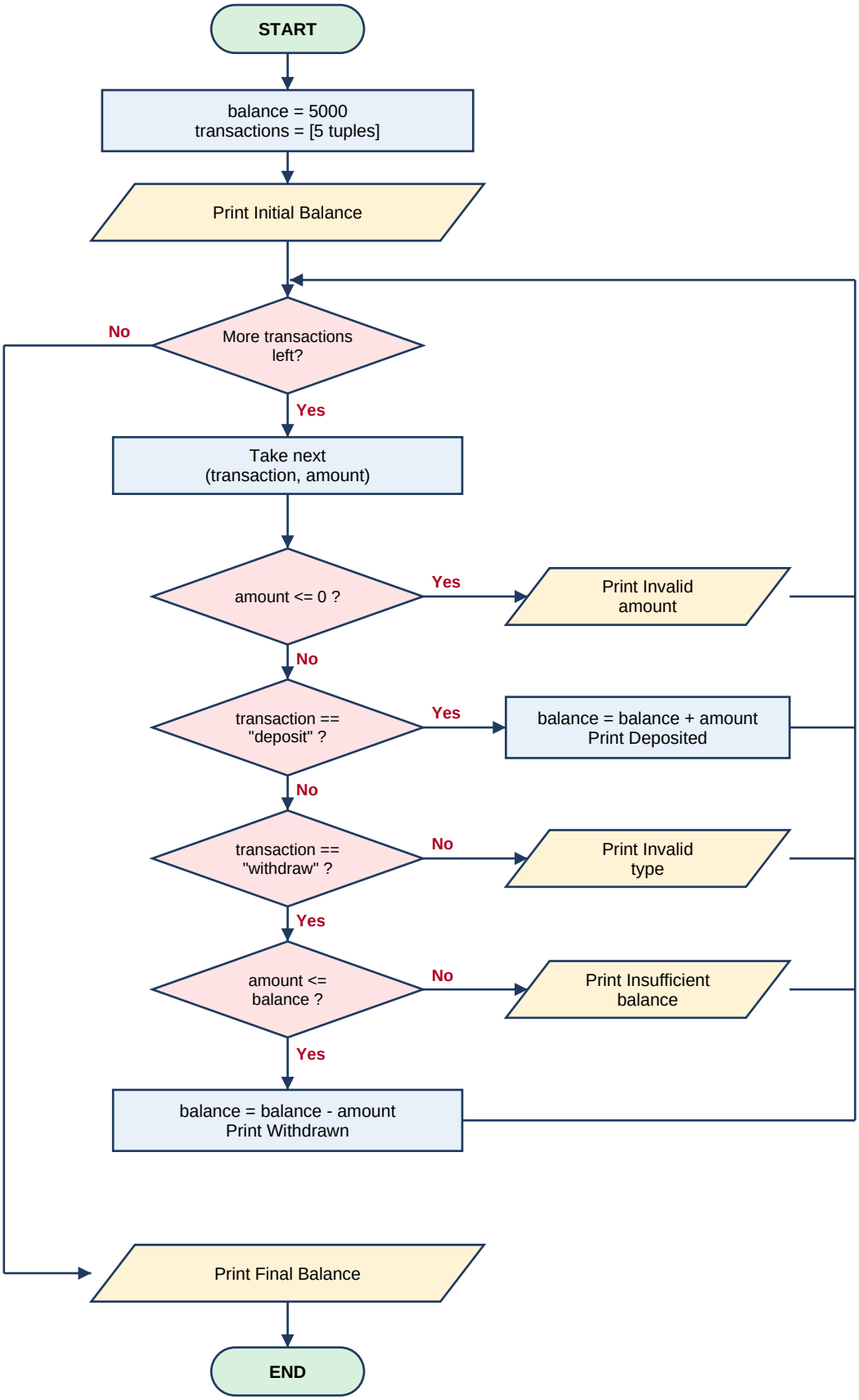
3. Assumptions

- The initial account balance is Rs. 5000.
- Each transaction is a tuple of the form (transaction type, amount); valid types are "deposit" and "withdraw".
- Transactions are processed strictly in the given order.
- A transaction with an amount less than or equal to zero is invalid.
- A withdrawal is allowed only if the amount is less than or equal to the current balance.
- Any transaction type other than "deposit" or "withdraw" is invalid.
- Amounts are whole numbers in rupees; the balance is never allowed to become negative.
- Test Case 1 contains only valid transactions, so no rejection occurs in this run.

4. Algorithm

1. Start.
2. Set balance = 5000 and create the list *transactions* of (type, amount) tuples.
3. Display the initial balance.
4. For each tuple (transaction, amount) in the list, repeat steps 5 to 9.
5. If amount <= 0, display "Invalid transaction amount" and go to the next transaction.
6. Else if transaction is "deposit": add amount to balance and display "Deposited: amount".
7. Else if transaction is "withdraw": if amount <= balance, subtract amount from balance and display "Withdrawn: amount"; otherwise display "Insufficient balance".
8. Else display "Invalid transaction type".
9. Move to the next transaction.
10. After all transactions are processed, display the final balance.
11. Stop.

5. Flowchart


START
balance = 5000
transactions = [5 tuples]
Print Initial Balance
More transactions left?
No
Yes
Take next (transaction, amount)
amount <= 0 ?
Yes
Print Invalid amount
No
transaction == "deposit" ?
Yes
balance = balance + amount
Print Deposited
No
transaction == "withdraw" ?
No
Print Invalid type
Yes
amount <= balance ?
No
Print Insufficient balance
Yes
balance = balance - amount
Print Withdrawn
Print Final Balance
END

6. Pseudocode

```
START
  SET balance <- 5000
  SET transactions <- [("deposit",2000), ("withdraw",1000),
                       ("withdraw",800), ("deposit",500),
                       ("withdraw",700)]
  PRINT "Initial Balance: Rs.", balance
  FOR EACH (transaction, amount) IN transactions DO
      IF amount <= 0 THEN
          PRINT "Invalid transaction amount"
      ELSE IF transaction = "deposit" THEN
          balance <- balance + amount
          PRINT "Deposited:", amount
      ELSE IF transaction = "withdraw" THEN
          IF amount <= balance THEN
              balance <- balance - amount
              PRINT "Withdrawn:", amount
          ELSE
              PRINT "Insufficient balance"
          END IF
      ELSE
          PRINT "Invalid transaction type"
      END IF
  END FOR
  PRINT "Final Balance: Rs.", balance
END
```

7. Python Source Code

File name: **tstcase1.py**

```python
# Case Study 1: ATM Transaction Simulation

balance = 5000
transactions = [
    ("deposit", 2000),
    ("withdraw", 1000),
    ("withdraw", 800),
    ("deposit", 500),
    ("withdraw", 700)
]
print("Initial Balance: Rs.", balance)

for transaction, amount in transactions:
    if amount <= 0:
        print("Invalid transaction amount")
    elif transaction == "deposit":
        balance = balance + amount
        print("Deposited:", amount)
    elif transaction == "withdraw":
        if amount <= balance:
            balance = balance - amount
            print("Withdrawn:", amount)
        else:
            print("Insufficient balance")
    else:
        print("Invalid transaction type")

print("Final Balance: Rs.", balance)
```

8. Test Cases

Test Case 1: A valid sequence of two deposits and three withdrawals starting from Rs. 5000.

Item	Details
Test case ID	TC-1
Initial balance	Rs. 5000
Input sequence	("deposit", 2000), ("withdraw", 1000), ("withdraw", 800), ("deposit", 500), ("withdraw", 700)
Conditions tested	Valid deposit, valid withdrawal where amount <= balance, loop and tuple unpacking
Expected final balance	Rs. 5000
Expected rejections	None

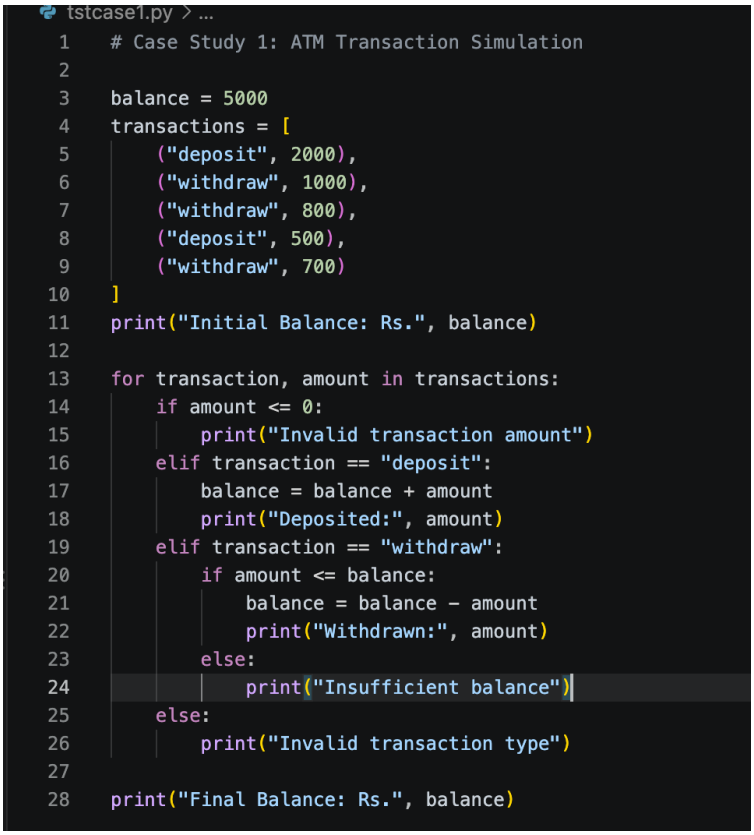
Step-by-step trace (expected result):

#	Transaction	Amount	Check	Action	Balance after
-	Initial	-	-	-	5000
1	deposit	2000	amount > 0	5000 + 2000	7000
2	withdraw	1000	1000 <= 7000	7000 - 1000	6000
3	withdraw	800	800 <= 6000	6000 - 800	5200
4	deposit	500	amount > 0	5200 + 500	5700
5	withdraw	700	700 <= 5700	5700 - 700	5000

Expected output	Actual output	Result
Final Balance: Rs. 5000	Final Balance: Rs. 5000	PASS

9. Output Screenshots

Screenshot 1: Source code **tstcase1.py** in VS Code.



Screenshot 2: Output of **tstcase1.py** in the VS Code terminal (Python 3.14.7).



10. Conclusion

The ATM Transaction Sequence Simulation correctly processed all five transactions of Test Case 1. Starting from Rs. 5000, the two deposits (Rs. 2000 and Rs. 500) and three withdrawals (Rs. 1000, Rs. 800 and Rs. 700) left a final balance of Rs. 5000, which matches the expected result. The program demonstrates the use of variables, a list of tuples, tuple unpacking, a for loop, and if / elif / else conditions (including a nested condition for the balance check). Because every transaction in this test case was valid, no rejection occurred; the invalid-amount, insufficient-balance and invalid-type branches are covered by further test cases.

Python Activity-Based Learning

Case Study 17: Student Performance Trend Analysis

Test Case 2

1. Problem Statement

Develop a Python program that analyses a student's performance across multiple examinations. The program stores the examination marks in a list, calculates the total and the average marks, compares the first and the last examination to find whether the performance has improved, declined or remained unchanged, and classifies the student (Excellent, Good, Average or Needs Improvement) according to the average marks.

2. Objectives

- To store examination marks in a list and traverse them using a **for** loop.
- To calculate the total and average marks using variables, arithmetic operators and the len() function.
- To identify the performance trend (improved, declined, unchanged) by comparing the first and last marks using list indexing.
- To classify the student using if / elif / else conditions on the average marks.
- To display the marks, average, trend and classification clearly.

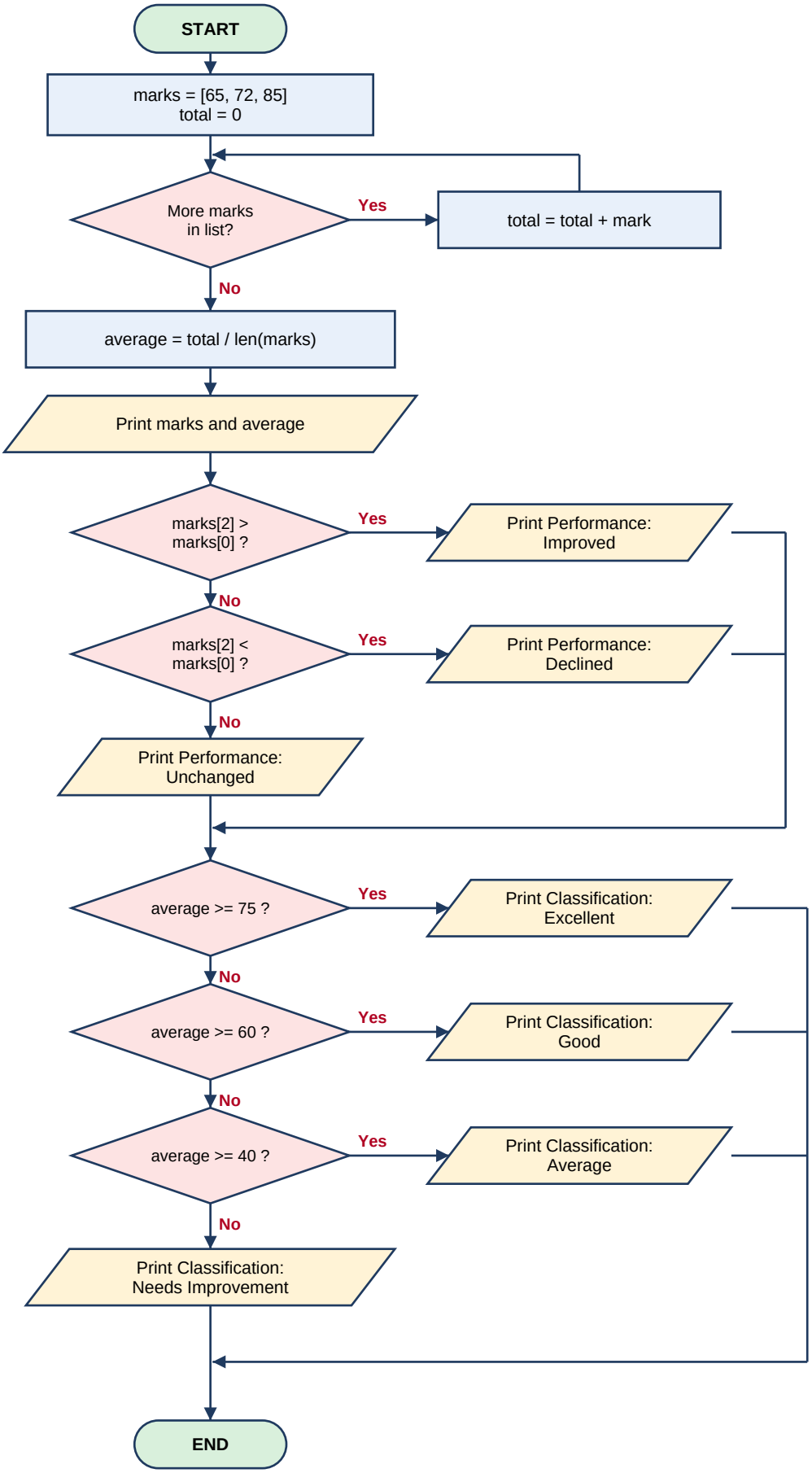
3. Assumptions

- The marks of three examinations are stored in the list marks = [65, 72, 85], in chronological order.
- All marks are out of 100 and are valid numbers.
- The trend is decided by comparing the last examination (marks[2]) with the first examination (marks[0]).
- Classification by average: 75 and above - Excellent; 60 to below 75 - Good; 40 to below 60 - Average; below 40 - Needs Improvement.
- The average is a decimal (float) value obtained with the division operator.

4. Algorithm

1. Start.
2. Create the list marks = [65, 72, 85] and set total = 0.
3. For each mark in the list, add the mark to total.
4. Calculate average = total / number of marks.
5. Display the examination marks and the average marks.
6. If marks[2] > marks[0], display "Performance: Improved".
7. Else if marks[2] < marks[0], display "Performance: Declined"; otherwise display "Performance: Unchanged".
8. If average >= 75, display "Classification: Excellent".
9. Else if average >= 60, display "Classification: Good".
10. Else if average >= 40, display "Classification: Average"; otherwise display "Classification: Needs Improvement".
11. Stop.

5. Flowchart


START
marks = [65, 72, 85]
total = 0
More marks in list?
Yes
total = total + mark
No
average = total / len(marks)
Print marks and average
marks[2] > marks[0] ?
Yes
Print Performance: Improved
No
marks[2] < marks[0] ?
Yes
Print Performance: Declined
No
Print Performance: Unchanged
average >= 75 ?
Yes
Print Classification: Excellent
No
average >= 60 ?
Yes
Print Classification: Good
No
average >= 40 ?
Yes
Print Classification: Average
No
Print Classification: Needs Improvement
END

6. Pseudocode

```
START
  SET marks <- [65, 72, 85]
  SET total <- 0
  FOR EACH mark IN marks DO
      total <- total + mark
  END FOR
  average <- total / LENGTH(marks)
  PRINT "Examination Marks:", marks
  PRINT "Average Marks:", average
  IF marks[2] > marks[0] THEN
      PRINT "Performance: Improved"
  ELSE IF marks[2] < marks[0] THEN
      PRINT "Performance: Declined"
  ELSE
      PRINT "Performance: Unchanged"
  END IF
  IF average >= 75 THEN
      PRINT "Classification: Excellent"
  ELSE IF average >= 60 THEN
      PRINT "Classification: Good"
  ELSE IF average >= 40 THEN
      PRINT "Classification: Average"
  ELSE
      PRINT "Classification: Needs Improvement"
  END IF
END
```

7. Python Source Code

File name: **case22.pyt**

```
 1  # Case Study 2: Student Performance Trend Analysis
 2
 3  marks = [65, 72, 85]
 4  total = 0
 5  for mark in marks:
 6      total = total + mark
 7  average = total / len(marks)
 8  print("Examination Marks:", marks)
 9  print("Average Marks:", average)
10  if marks[2] > marks[0]:
11      print("Performance: Improved")
12  elif marks[2] < marks[0]:
13      print("Performance: Declined")
14  else:
15      print("Performance: Unchanged")
16
17  if average >= 75:
18      print("Classification: Excellent")
19  elif average >= 60:
20      print("Classification: Good")
21  elif average >= 40:
22      print("Classification: Average")
23  else:
24      print("Classification: Needs Improvement")
```

8. Test Cases

Test Case 2: Marks of three examinations show an upward trend with an average in the Good range.

Item	Details
Test case ID	TC-2
Input (marks list)	[65, 72, 85]
Conditions tested	List traversal with a loop, average calculation, first-vs-last comparison (improved), classification (Good)
Expected output	Average = 74.0, Performance: Improved, Classification: Good

Step-by-step trace (expected result):

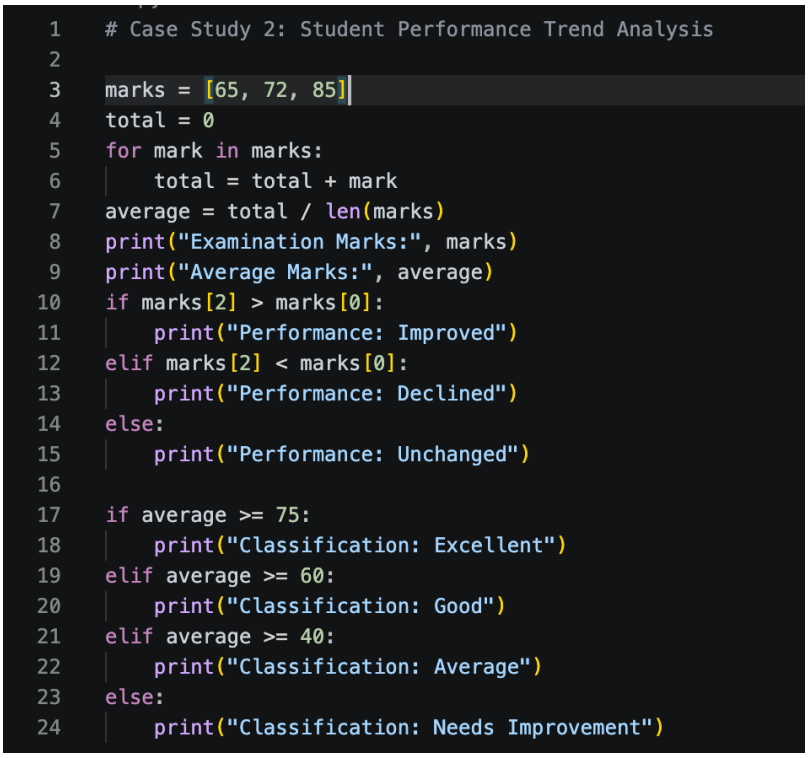
Step	mark	Calculation	total
1	65	0 + 65	65
2	72	65 + 72	137
3	85	137 + 85	222

Check	Evaluation	Result
Average	222 / 3	74.0
Trend	marks[2] > marks[0] (85 > 65)	Improved
average >= 75	74.0 >= 75 is False	next condition
average >= 60	74.0 >= 60 is True	Good

Expected output	Actual output	Result
Average Marks: 74.0, Improved, Good	Average Marks: 74.0, Improved, Good	PASS

9. Output Screenshots

Screenshot 1: Source code **case22.pyt** in VS Code.

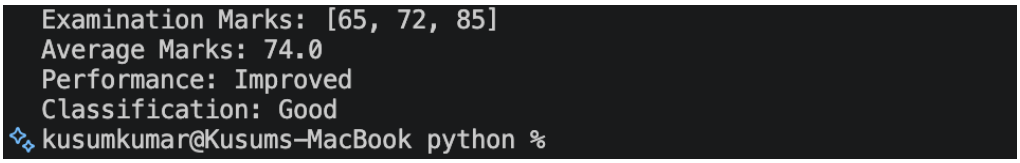


```
# Case Study 2: Student Performance Trend Analysis

marks = [65, 72, 85]
total = 0
for mark in marks:
    total = total + mark
average = total / len(marks)
print("Examination Marks:", marks)
print("Average Marks:", average)
if marks[2] > marks[0]:
    print("Performance: Improved")
elif marks[2] < marks[0]:
    print("Performance: Declined")
else:
    print("Performance: Unchanged")

if average >= 75:
    print("Classification: Excellent")
elif average >= 60:
    print("Classification: Good")
elif average >= 40:
    print("Classification: Average")
else:
    print("Classification: Needs Improvement")
```

Screenshot 2: Output of **case22.pyt** in the VS Code terminal (Python 3.14.7).



```
Examination Marks: [65, 72, 85]
Average Marks: 74.0
Performance: Improved
Classification: Good
kusumkumar@Kusums-MacBook python %
```

10. Conclusion

The Student Performance Trend Analysis program worked correctly for Test Case 2. For the marks [65, 72, 85] it calculated a total of 222 and an average of 74.0, identified the performance as Improved because the last examination score (85) is higher than the first (65), and classified the student as Good because the average lies between 60 and 75. The program demonstrates the use of lists, indexing, a for loop, arithmetic operators, and if / elif / else conditions. The Declined, Unchanged, Excellent, Average and Needs Improvement branches are covered by further test cases.

Python Activity-Based Learning

Case Study 26: Nested Loop Examination Seating Generator

Test Case 3

1. Problem Statement

Develop a Python program that generates an examination seating arrangement for a classroom with a fixed number of rows and columns. Using nested loops, the program visits every seat position (row, column), marks the seats that are reserved, marks all remaining seats as available, and prints the arrangement row by row as a seating grid.

2. Objectives

- To use nested **for** loops with range() to visit every row and column of a classroom.
- To represent each seat position as a tuple (row, column) and store reserved seats in a list of tuples.
- To use the membership operator **in** with an if / else condition to identify reserved seats.
- To control output formatting with print(end=" ") and print() to produce a neat seating grid.
- To generate a seating layout that can be read directly by invigilators.

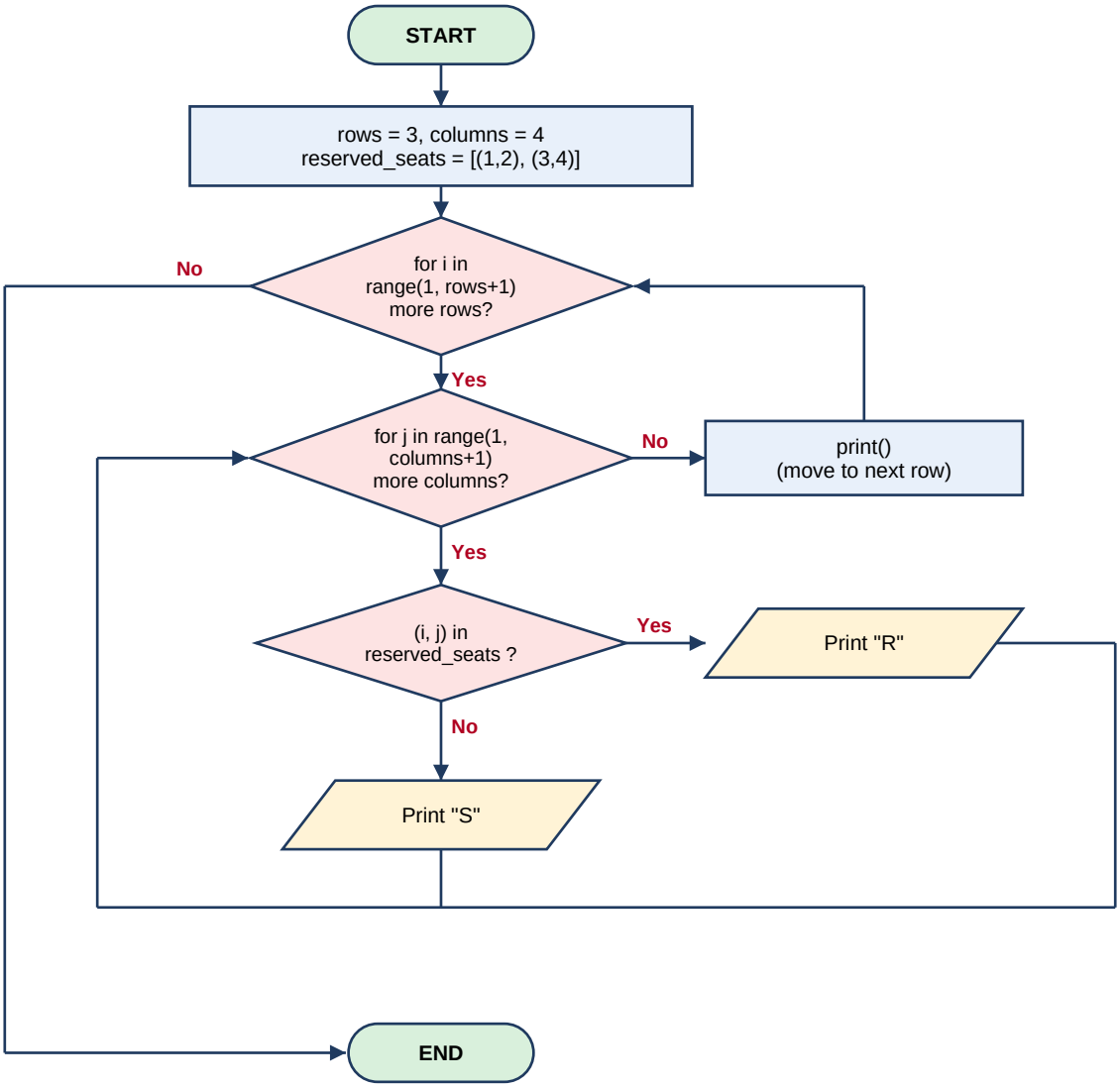
3. Assumptions

- The classroom has 3 rows and 4 columns, so the fixed capacity is 3 x 4 = 12 positions.
- Rows and columns are numbered starting from 1; a seat is identified by the tuple (row, column).
- The reserved seats are (1, 2) and (3, 4), stored in the list reserved_seats.
- "R" denotes a reserved seat and "S" denotes a seat available for allocation.
- Seats are printed row by row, with one space between seats.

4. Algorithm

1. Start.
2. Set rows = 3, columns = 4 and reserved_seats = [(1, 2), (3, 4)].
3. Repeat for each row i from 1 to rows (outer loop).
4. Repeat for each column j from 1 to columns (inner loop).
5. If the tuple (i, j) is present in reserved_seats, print "R"; otherwise print "S" (on the same line).
6. After all columns of the row are processed, print a new line.
7. Move to the next row; when all rows are finished, go to step 8.
8. Stop.

5. Flowchart



6. Pseudocode

```
START
  SET rows <- 3
  SET columns <- 4
  SET reserved_seats <- [(1,2), (3,4)]
  FOR i <- 1 TO rows DO
      FOR j <- 1 TO columns DO
          IF (i, j) IS IN reserved_seats THEN
              PRINT "R" (stay on the same line)
          ELSE
              PRINT "S" (stay on the same line)
          END IF
      END FOR
      PRINT new line
  END FOR
END
```

7. Python Source Code

File name: **case3.pyt**

```
# Case Study 3: Examination Seating Generator

rows = 3
columns = 4
reserved_seats = [(1, 2), (3, 4)]
for i in range(1, rows + 1):
    for j in range(1, columns + 1):
        if (i, j) in reserved_seats:
            print("R", end=" ")
        else:
            print("S", end=" ")
    print()
```

8. Test Cases

Test Case 3: A classroom of 3 rows and 4 columns with two reserved seats, (1, 2) and (3, 4).

Item	Details
Test case ID	TC-3
Input	rows = 3, columns = 4, reserved_seats = [(1, 2), (3, 4)]
Conditions tested	Nested loops, tuple membership test with **in**, if / else, formatted grid output
Expected output	3 rows of 4 seats; R at row 1 column 2 and at row 3 column 4; all other seats S

Seat-by-seat trace (expected result):

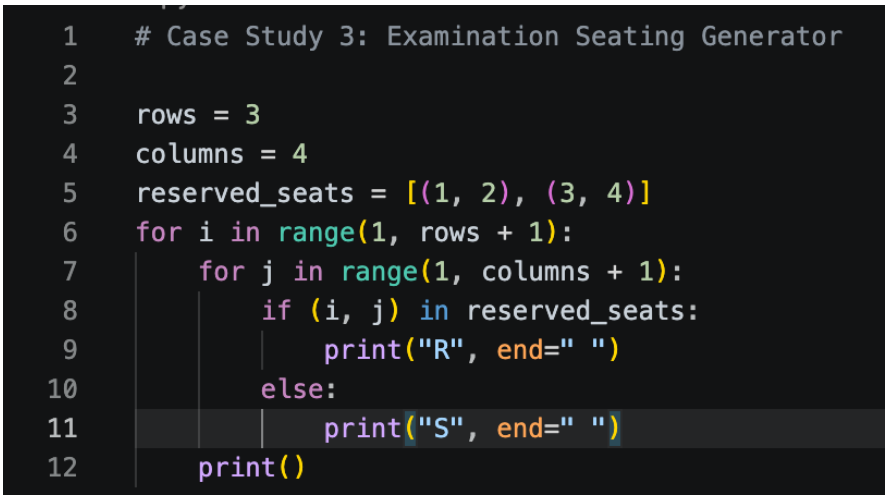
Row (i)	Col 1 (1,j)	Col 2	Col 3	Col 4	Printed line
1	S	R (1,2) reserved	S	S	S R S S
2	S	S	S	S	S S S S
3	S	S	S	R (3,4) reserved	S S S R

Check	Expected	Actual
Total positions printed	12	12
Reserved seats (R)	2	2
Available seats (S)	10	10

Expected output	Actual output	Result
S R S S / S S S S / S S S R	S R S S / S S S S / S S S R	PASS

9. Output Screenshots

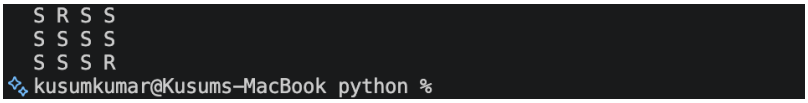
Screenshot 1: Source code **case3.pyt** in VS Code.



```
# Case Study 3: Examination Seating Generator

rows = 3
columns = 4
reserved_seats = [(1, 2), (3, 4)]
for i in range(1, rows + 1):
    for j in range(1, columns + 1):
        if (i, j) in reserved_seats:
            print("R", end=" ")
        else:
            print("S", end=" ")
    print()
```

Screenshot 2: Output of **case3.pyt** in the VS Code terminal (Python 3.14.7).



```
S R S S
S S S S
S S S R
kusumkumar@Kusums-MacBook python %
```

10. Conclusion

The Nested Loop Examination Seating Generator produced the correct seating layout for Test Case 3. For a classroom of 3 rows and 4 columns, the nested loops visited all 12 seat positions, marked the reserved seats (1, 2) and (3, 4) as R, and marked the remaining 10 seats as S, giving the grid S R S S / S S S S / S S S R, which matches the expected output. The program demonstrates nested for loops, range(), tuples, the membership operator, if / else conditions and formatted output using print(end=" ") and print().

Python Activity-Based Learning

Case Study 42: Student Marks Matrix Analysis

Test Case 4

1. Problem Statement

Develop a Python program that analyses a matrix of student marks across several subjects. The marks are stored as a nested list in which every inner list holds the marks of one student. For each student the program calculates the total and the average marks, and finally it identifies the top performer, that is, the student with the highest total.

2. Objectives

- To represent a marks matrix as a nested list (a list of lists).
- To traverse the matrix using nested **for** loops.
- To calculate row-wise totals and averages using variables, operators and len().
- To store the totals in a list and find the maximum using max() and index().
- To identify and display the top performer along with the highest total.

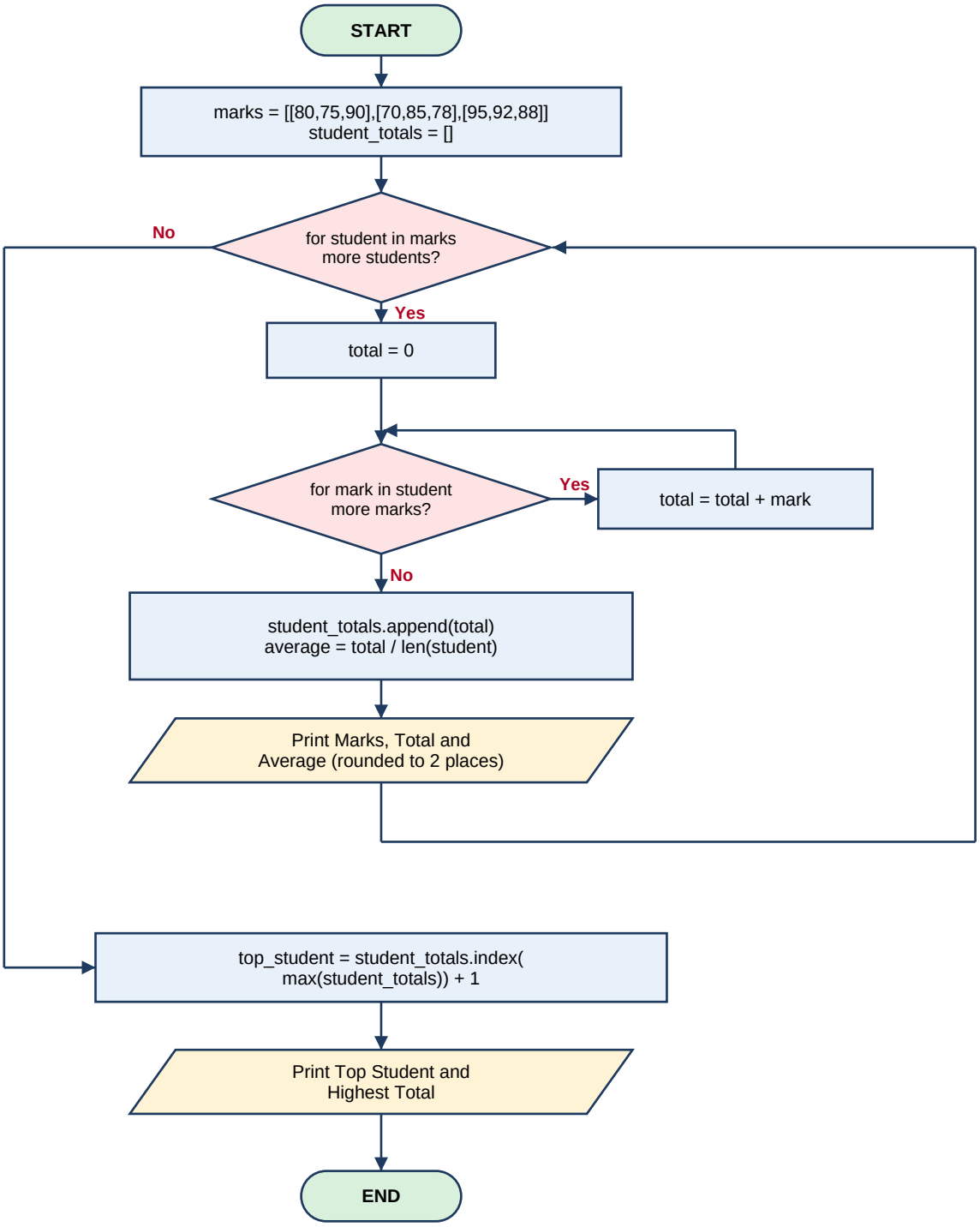
3. Assumptions

- The matrix has 3 students (rows) and 3 subjects (columns); each row holds one student's marks.
- All marks are out of 100 and are valid numbers.
- Students are numbered from 1 in the order in which their rows appear in the matrix.
- The average is rounded to 2 decimal places using round().
- If two students have the same highest total, the first one in the matrix is reported as the top student.

4. Algorithm

1. Start.
2. Create the nested list marks and an empty list student_totals.
3. For each student (row) in marks, set total = 0.
4. For each mark in that student's row, add the mark to total.
5. Append total to student_totals.
6. Calculate average = total / number of subjects.
7. Display the student's marks, total and average (rounded to 2 decimals).
8. Repeat steps 3 to 7 for every student.
9. Find the highest total using max() and its position using index(); top student = position + 1.
10. Display the top student and the highest total.
11. Stop.

5. Flowchart


START
marks = [[80,75,90],[70,85,78],[95,92,88]]
student_totals = []
for student in marks
more students?
No
Yes
total = 0
for mark in student
more marks?
Yes
total = total + mark
No
student_totals.append(total)
average = total / len(student)
Print Marks, Total and
Average (rounded to 2 places)
top_student = student_totals.index(
max(student_totals)) + 1
Print Top Student and
Highest Total
END

6. Pseudocode

```
START
  SET marks <- [[80,75,90], [70,85,78], [95,92,88]]
  SET student_totals <- empty list
  FOR EACH student IN marks DO
      SET total <- 0
      FOR EACH mark IN student DO
          total <- total + mark
      END FOR
      APPEND total TO student_totals
      average <- total / LENGTH(student)
      PRINT "Marks:", student
      PRINT "Total:", total
      PRINT "Average:", ROUND(average, 2)
  END FOR
  top_student <- INDEX OF MAX(student_totals) + 1
  PRINT "Top Student: Student", top_student
  PRINT "Highest Total:", MAX(student_totals)
END
```

7. Python Source Code

File name: **case4.pyt**

```
# Case Study 4: Student Marks Matrix Analysis

marks = [
    [80, 75, 90],
    [70, 85, 78],
    [95, 92, 88]
]

student_totals = []

for student in marks:
    total = 0

    for mark in student:
        total = total + mark

    student_totals.append(total)

    average = total / len(student)

    print("Marks:", student)
    print("Total:", total)
    print("Average:", round(average, 2))

top_student = student_totals.index(max(student_totals)) + 1

print("Top Student: Student", top_student)
print("Highest Total:", max(student_totals))
```

8. Test Cases

Test Case 4: A 3 x 3 marks matrix for three students; Student 3 has the highest total.

Item	Details
Test case ID	TC-4
Input (marks matrix)	[[80, 75, 90], [70, 85, 78], [95, 92, 88]]
Conditions tested	Nested list traversal, row-wise total and average, max() and index() to find the top student
Expected output	Totals 245, 233, 275; averages 81.67, 77.67, 91.67; Top Student: Student 3; Highest Total: 275

Row-wise calculation (expected result):

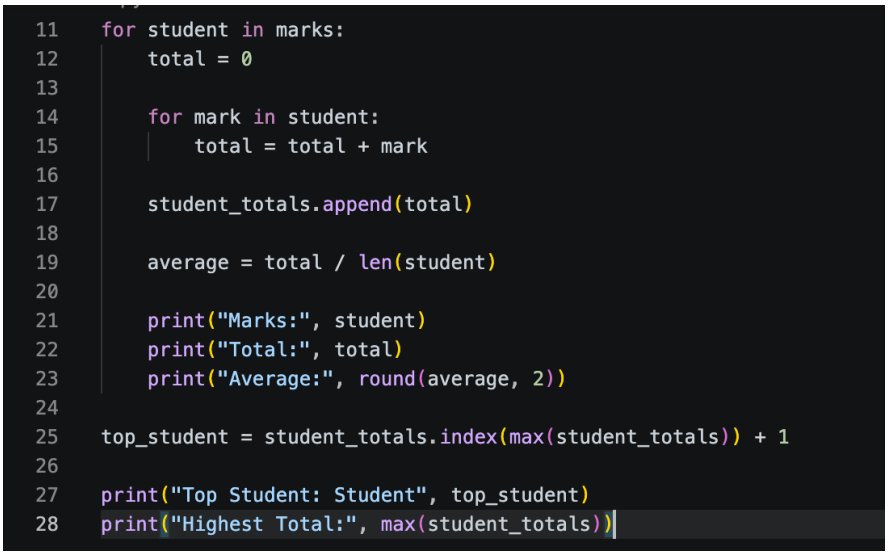
Student	Marks	Total	Average (total / 3)	Rounded
Student 1	[80, 75, 90]	80 + 75 + 90 = 245	245 / 3 = 81.666...	81.67
Student 2	[70, 85, 78]	70 + 85 + 78 = 233	233 / 3 = 77.666...	77.67
Student 3	[95, 92, 88]	95 + 92 + 88 = 275	275 / 3 = 91.666...	91.67

Check	Evaluation	Result
student_totals	[245, 233, 275]	list built
max(student_totals)	max of 245, 233, 275	275
index of 275	position 2 (counting from 0)	2
top_student	2 + 1	Student 3

Expected output	Actual output	Result
Top Student: Student 3, Highest Total: 275	Top Student: Student 3, Highest Total: 275	PASS

9. Output Screenshots

Screenshot 1: Source code **case4.pyt** in VS Code (lines 11 to 28 visible).



```
11  for student in marks:
12      total = 0
13
14      for mark in student:
15          total = total + mark
16
17      student_totals.append(total)
18
19      average = total / len(student)
20
21      print("Marks:", student)
22      print("Total:", total)
23      print("Average:", round(average, 2))
24
25  top_student = student_totals.index(max(student_totals)) + 1
26
27  print("Top Student: Student", top_student)
28  print("Highest Total:", max(student_totals))
```

Screenshot 2: Output of **case4.pyt** in the VS Code terminal (Python 3.14.7).



```
Marks: [80, 75, 90]
Total: 245
Average: 81.67
Marks: [70, 85, 78]
Total: 233
Average: 77.67
Marks: [95, 92, 88]
Total: 275
Average: 91.67
Top Student: Student 3
Highest Total: 275
kusumkumar@Kusums-MacBook python %
```

10. Conclusion

The Student Marks Matrix Analysis program produced the correct results for Test Case 4. For the matrix [[80, 75, 90], [70, 85, 78], [95, 92, 88]] it calculated totals of 245, 233 and 275 and averages of 81.67, 77.67 and 91.67, then correctly reported Student 3 as the top performer with the highest total of 275. The program demonstrates nested lists, nested for loops, list methods such as append() and index(), the built-in functions max(), len() and round(), and formatted output.

Python Activity-Based Learning

Case Study 51: Bus Passenger Occupancy Simulation

Test Case 5

1. Problem Statement

Develop a Python program that simulates the number of passengers inside a bus as it travels through several stops. At each stop a number of passengers board and a number alight. The program calculates the passenger count after every stop, rejects a stop whose alighting count is greater than the passengers present, and flags any stop at which the bus capacity is exceeded.

2. Objectives

- To store the boarding and alighting counts of each stop as tuples in a list.
- To use a **for** loop with enumerate() and tuple unpacking to process the stops in order.
- To update the passenger count using variables and arithmetic operators.
- To validate the alighting count and the bus capacity using if / else conditions.
- To use **continue** to skip an invalid stop, and to display the passenger count at each stop and at the end.

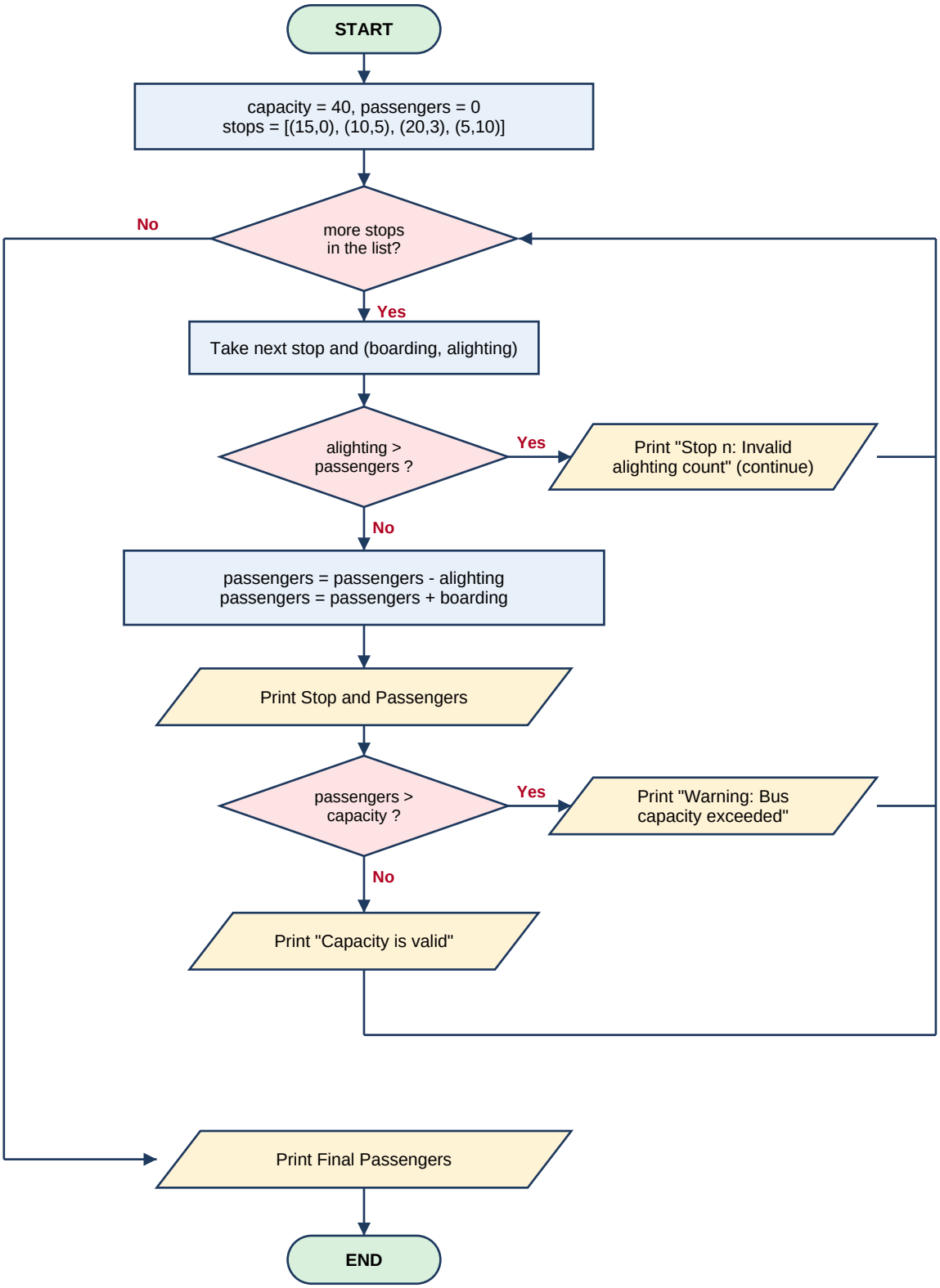
3. Assumptions

- The bus capacity is 40 passengers and the bus is empty at the start (passengers = 0).
- Each stop is a tuple (boarding, alighting), and stops are numbered from 1 in travel order.
- At every stop the passengers alight first and then the new passengers board.
- A stop is invalid if the alighting count is greater than the passengers currently on the bus; such a stop is skipped without changing the count.
- A capacity violation occurs when the passenger count after a stop is greater than 40.

4. Algorithm

1. Start.
2. Set capacity = 40, passengers = 0 and create the list stops of (boarding, alighting) tuples.
3. For each stop (numbered from 1), unpack the tuple into boarding and alighting.
4. If alighting is greater than passengers, display "Invalid alighting count" and go to the next stop.
5. Subtract alighting from passengers, then add boarding to passengers.
6. Display the stop number and the passenger count.
7. If passengers is greater than capacity, display "Warning: Bus capacity exceeded"; otherwise display "Capacity is valid".
8. Repeat steps 3 to 7 for every stop.
9. Display the final number of passengers.
10. Stop.

5. Flowchart


START
capacity = 40, passengers = 0
stops = [(15,0), (10,5), (20,3), (5,10)]
more stops in the list?
No
Yes
Take next stop and (boarding, alighting)
alighting > passengers ?
Yes
Print "Stop n: Invalid alighting count" (continue)
No
passengers = passengers - alighting
passengers = passengers + boarding
Print Stop and Passengers
passengers > capacity ?
Yes
Print "Warning: Bus capacity exceeded"
No
Print "Capacity is valid"
Print Final Passengers
END

6. Pseudocode

```
START
  SET capacity <- 40
  SET passengers <- 0
  SET stops <- [(15,0), (10,5), (20,3), (5,10)]
  FOR EACH stop NUMBER (from 1) WITH (boarding, alighting) IN stops DO
      IF alighting > passengers THEN
          PRINT "Stop", stop, ": Invalid alighting count"
          CONTINUE with the next stop
      END IF
      passengers <- passengers - alighting
      passengers <- passengers + boarding
      PRINT "Stop:", stop
      PRINT "Passengers:", passengers
      IF passengers > capacity THEN
          PRINT "Warning: Bus capacity exceeded"
      ELSE
          PRINT "Capacity is valid"
      END IF
  END FOR
  PRINT "Final Passengers:", passengers
END
```

7. Python Source Code

File name: **case5.pyt**

```
# Case Study 5: Bus Passenger Occupancy

capacity = 40
passengers = 0
stops = [
    (15, 0),
    (10, 5),
    (20, 3),
    (5, 10)
]
for stop, (boarding, alighting) in enumerate(stops, start=1):
    if alighting > passengers:
        print("Stop", stop, ": Invalid alighting count")
        continue
    passengers = passengers - alighting
    passengers = passengers + boarding
    print("Stop:", stop)
    print("Passengers:", passengers)

    if passengers > capacity:
        print("Warning: Bus capacity exceeded")
    else:
        print("Capacity is valid")
print("Final Passengers:", passengers)
```

8. Test Cases

Test Case 5: A bus with capacity 40 travels through four valid stops; the passenger count never exceeds the capacity.

Item	Details
Test case ID	TC-5
Input	capacity = 40, stops = [(15, 0), (10, 5), (20, 3), (5, 10)] (boarding, alighting)
Conditions tested	enumerate() with tuple unpacking, alighting validation, passenger update, capacity check
Expected output	Passengers after stops: 15, 20, 37, 32; capacity valid at every stop; Final Passengers: 32

Stop-by-stop trace (expected result):

Stop	Boarding	Alighting	Alighting > passengers?	Calculation	Passengers	Capacity
1	15	0	0 > 0 : No	0 - 0 + 15	15	Valid
2	10	5	5 > 15 : No	15 - 5 + 10	20	Valid
3	20	3	3 > 20 : No	20 - 3 + 20	37	Valid
4	5	10	10 > 37 : No	37 - 10 + 5	32	Valid

Check	Expected	Actual
Invalid stops rejected	0	0
Capacity warnings	0 (maximum 37 is not above 40)	0
Final Passengers	32	32

Expected output	Actual output	Result
Final Passengers: 32	Final Passengers: 32	PASS

9. Output Screenshots

Screenshot 1: Source code **case5.pyt** in VS Code.

```python
# Case Study 5: Bus Passenger Occupancy

capacity = 40
passengers = 0
stops = [
    (15, 0),
    (10, 5),
    (20, 3),
    (5, 10)
]
for stop, (boarding, alighting) in enumerate(stops, start=1):
    if alighting > passengers:
        print("Stop", stop, ": Invalid alighting count")
        continue
    passengers = passengers - alighting
    passengers = passengers + boarding
    print("Stop:", stop)
    print("Passengers:", passengers)

    if passengers > capacity:
        print("Warning: Bus capacity exceeded")
    else:
        print("Capacity is valid")
print("Final Passengers:", passengers)
```

Screenshot 2: Output of **case5.pyt** in the VS Code terminal (Python 3.14.7).

```
Stop: 1
Passengers: 15
Capacity is valid
Stop: 2
Passengers: 20
Capacity is valid
Stop: 3
Passengers: 37
Capacity is valid
Stop: 4
Passengers: 32
Capacity is valid
Final Passengers: 32
kusumkumar@Kusums-MacBook python %
```

10. Conclusion

The Bus Passenger Occupancy Simulation produced the correct result for Test Case 5. Starting with an empty bus, the passenger counts after the four stops were 15, 20, 37 and 32, the capacity of 40 was never exceeded and no stop was rejected, so the final count of 32 passengers matches the expected output. The program demonstrates a list of tuples, enumerate() with tuple unpacking, the continue statement, arithmetic operators and if / else conditions. The invalid alighting count and capacity exceeded branches are covered by further test cases.

Python Activity-Based Learning

Case Study 53: Food Waste Monitoring System

Test Case 6

1. Problem Statement

Develop a Python program that monitors food wastage in a canteen over several days. For each day, the number of meals prepared and the number of meals served are compared to find the meals wasted. The program calculates the waste percentage for every day, displays the day-wise report and identifies the days on which the waste percentage exceeds the acceptable limit.

2. Objectives

- To store the daily meals prepared and meals served in two lists.
- To traverse both lists together using a **for** loop with range(), len() and index positions.
- To calculate meals wasted and waste percentage using variables and arithmetic operators.
- To compare each day's waste percentage with the acceptable limit using if / else.
- To display a clear day-wise report using round() and formatted print statements.

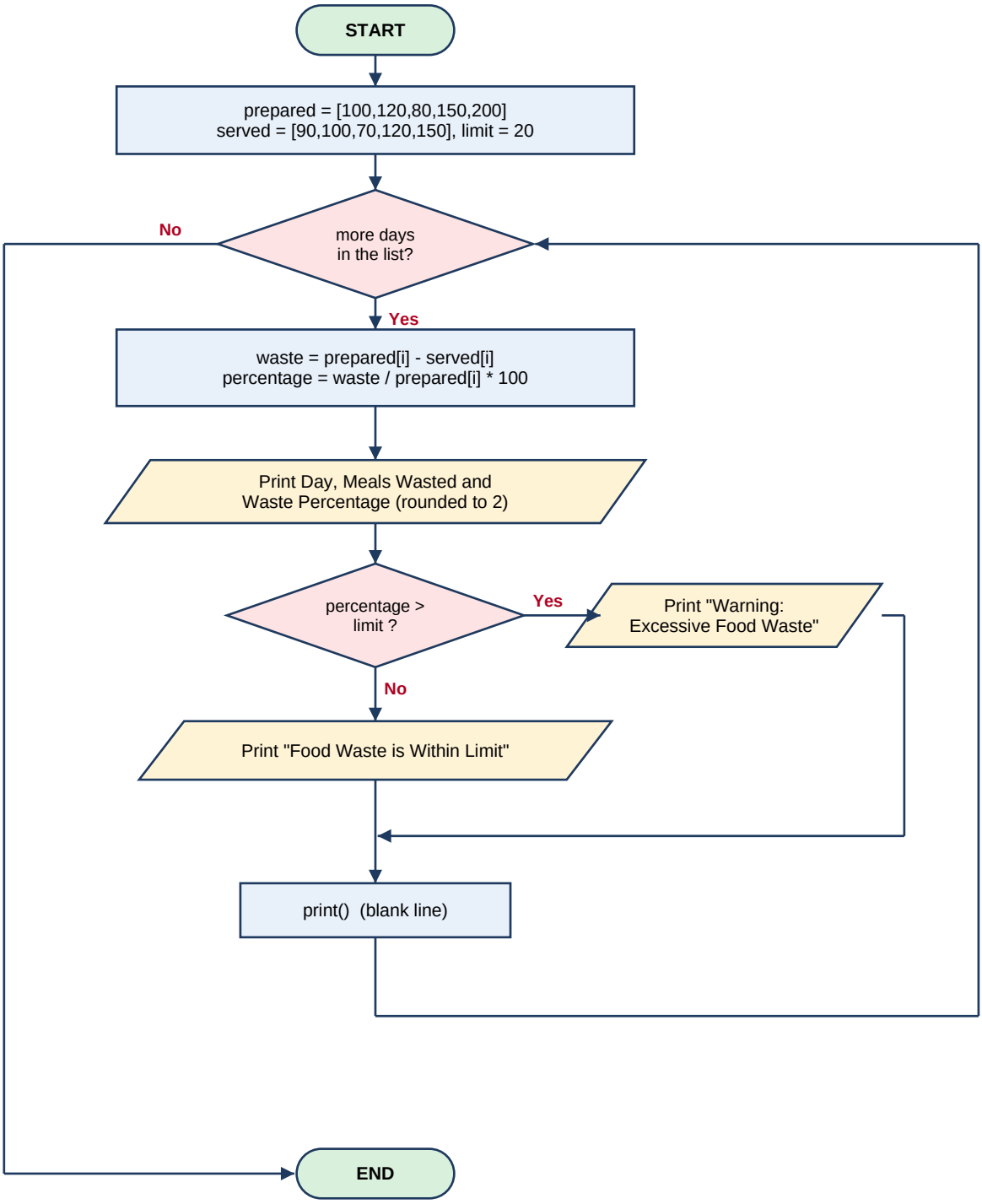
3. Assumptions

- Data for five days is given: meals prepared = [100, 120, 80, 150, 200] and meals served = [90, 100, 70, 120, 150].
- Meals wasted = meals prepared - meals served (all unserved meals are treated as wasted).
- Waste percentage = (meals wasted / meals prepared) x 100, rounded to 2 decimal places.
- The acceptable limit is 20%; waste is excessive only when the percentage is strictly greater than 20.
- Days are numbered from 1; both lists have the same length and meals prepared is never zero.

4. Algorithm

1. Start.
2. Create the lists prepared and served, and set limit = 20.
3. For each day index i from 0 to the number of days - 1, repeat steps 4 to 8.
4. Calculate waste = prepared[i] - served[i].
5. Calculate percentage = waste / prepared[i] x 100.
6. Display the day number (i + 1), the meals wasted and the waste percentage rounded to 2 decimals.
7. If percentage is greater than limit, display "Warning: Excessive Food Waste"; otherwise display "Food Waste is Within Limit".
8. Print a blank line and move to the next day.
9. Stop.

5. Flowchart


START
prepared = [100,120,80,150,200]
served = [90,100,70,120,150], limit = 20
more days
in the list?
No
Yes
waste = prepared[i] - served[i]
percentage = waste / prepared[i] * 100
Print Day, Meals Wasted and
Waste Percentage (rounded to 2)
percentage >
limit ?
Yes
Print "Warning:
Excessive Food Waste"
No
Print "Food Waste is Within Limit"
print() (blank line)
END

6. Pseudocode

```
START
  SET prepared <- [100, 120, 80, 150, 200]
  SET served <- [90, 100, 70, 120, 150]
  SET limit <- 20
  FOR i <- 0 TO LENGTH(prepared) - 1 DO
      waste <- prepared[i] - served[i]
      percentage <- waste / prepared[i] * 100
      PRINT "Day:", i + 1
      PRINT "Meals Wasted:", waste
      PRINT "Waste Percentage:", ROUND(percentage, 2), "%"
      IF percentage > limit THEN
          PRINT "Warning: Excessive Food Waste"
      ELSE
          PRINT "Food Waste is Within Limit"
      END IF
      PRINT blank line
  END FOR
END
```

7. Python Source Code

File name: **case6.pyt**

```
 1  # Case Study 6: Food Waste Monitoring
 2
 3  prepared = [100, 120, 80, 150, 200]
 4  served = [90, 100, 70, 120, 150]
 5
 6  limit = 20
 7
 8  for i in range(len(prepared)):
 9
10      waste = prepared[i] - served[i]
11      percentage = waste / prepared[i] * 100
12
13      print("Day:", i + 1)
14      print("Meals Wasted:", waste)
15      print("Waste Percentage:", round(percentage, 2), "%")
16
17      if percentage > limit:
18          print("Warning: Excessive Food Waste")
19      else:
20          print("Food Waste is Within Limit")
21
22      print()
```

8. Test Cases

Test Case 6: Five days of meal data with a 20% waste limit; Day 4 is exactly at the limit and Day 5 exceeds it.

Item	Details
Test case ID	TC-6
Input	prepared = [100, 120, 80, 150, 200], served = [90, 100, 70, 120, 150], limit = 20
Conditions tested	Index-based list traversal, waste and percentage calculation, boundary value (exactly 20%), warning branch
Expected output	Wastes 10, 20, 10, 30, 50; percentages 10.0, 16.67, 12.5, 20.0, 25.0; warning only on Day 5

Day-wise calculation (expected result):

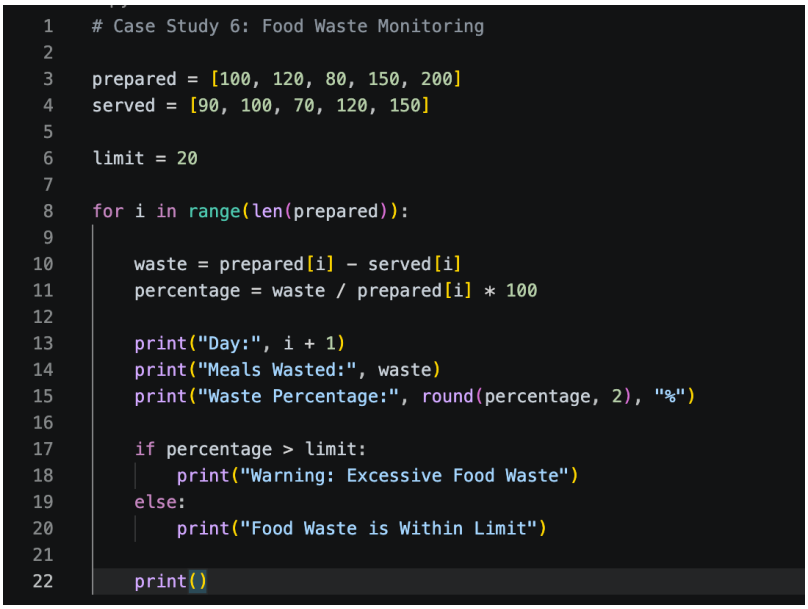
Day	Prepared	Served	Waste	Percentage	Rounded	> 20 ?	Message
1	100	90	10	10/100 x 100	10.0	No	Within Limit
2	120	100	20	20/120 x 100	16.67	No	Within Limit
3	80	70	10	10/80 x 100	12.5	No	Within Limit
4	150	120	30	30/150 x 100	20.0	No (equal)	Within Limit
5	200	150	50	50/200 x 100	25.0	Yes	Warning

Check	Expected	Actual
Days processed	5	5
Days exceeding the 20% limit	1 (Day 5)	1 (Day 5)
Highest waste percentage	25.0 % (Day 5)	25.0 % (Day 5)

Expected output	Actual output	Result
Day 5: Waste Percentage 25.0 %, Warning: Excessive Food Waste	Day 5: Waste Percentage 25.0 %, Warning: Excessive Food Waste	PASS

9. Output Screenshots

Screenshot 1: Source code **case6.pyt** in VS Code.



```python
# Case Study 6: Food Waste Monitoring

prepared = [100, 120, 80, 150, 200]
served = [90, 100, 70, 120, 150]

limit = 20

for i in range(len(prepared)):

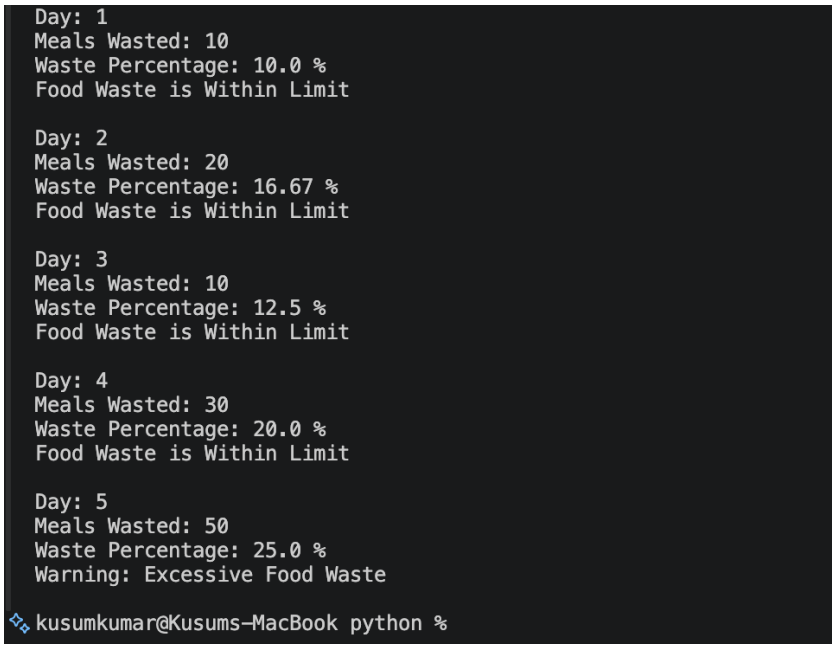
    waste = prepared[i] - served[i]
    percentage = waste / prepared[i] * 100

    print("Day:", i + 1)
    print("Meals Wasted:", waste)
    print("Waste Percentage:", round(percentage, 2), "%")

    if percentage > limit:
        print("Warning: Excessive Food Waste")
    else:
        print("Food Waste is Within Limit")

    print()
```

Screenshot 2: Output of **case6.pyt** in the VS Code terminal (Python 3.14.7).



```
Day: 1
Meals Wasted: 10
Waste Percentage: 10.0 %
Food Waste is Within Limit

Day: 2
Meals Wasted: 20
Waste Percentage: 16.67 %
Food Waste is Within Limit

Day: 3
Meals Wasted: 10
Waste Percentage: 12.5 %
Food Waste is Within Limit

Day: 4
Meals Wasted: 30
Waste Percentage: 20.0 %
Food Waste is Within Limit

Day: 5
Meals Wasted: 50
Waste Percentage: 25.0 %
Warning: Excessive Food Waste

kusumkumar@Kusums-MacBook python %
```

10. Conclusion

The Food Waste Monitoring System produced the correct report for Test Case 6. For the five days it calculated meals wasted of 10, 20, 10, 30 and 50, with waste percentages of 10.0, 16.67, 12.5, 20.0 and 25.0. Days 1 to 4 were reported as within the limit (Day 4 is exactly 20%, which is not above the limit), and only Day 5 was flagged with the warning for excessive food waste, as expected. The program demonstrates lists, index-based traversal with range() and len(), arithmetic operators, round() and if / else conditions.

Python Activity-Based Learning

Case Study 60: Manufacturing Quality Control Matrix

Test Case 7

1. Problem Statement

Develop a Python program for quality control in a manufacturing unit. The production data is stored as a matrix (nested list) in which every row holds the units produced and the defective units of one batch. For each batch the program calculates the defect percentage, displays a batch report, and decides whether the batch is accepted or requires a quality check by comparing the defect percentage with the permitted quality limit.

2. Objectives

- To represent production data as a nested list (matrix) of [units produced, defective units] rows.
- To access matrix elements using two-level indexing such as production[i][0] and production[i][1].
- To calculate the defect percentage of every batch using variables and arithmetic operators.
- To compare the defect percentage with the quality limit using if / else conditions.
- To display a clear batch-wise quality report using round() and print statements.

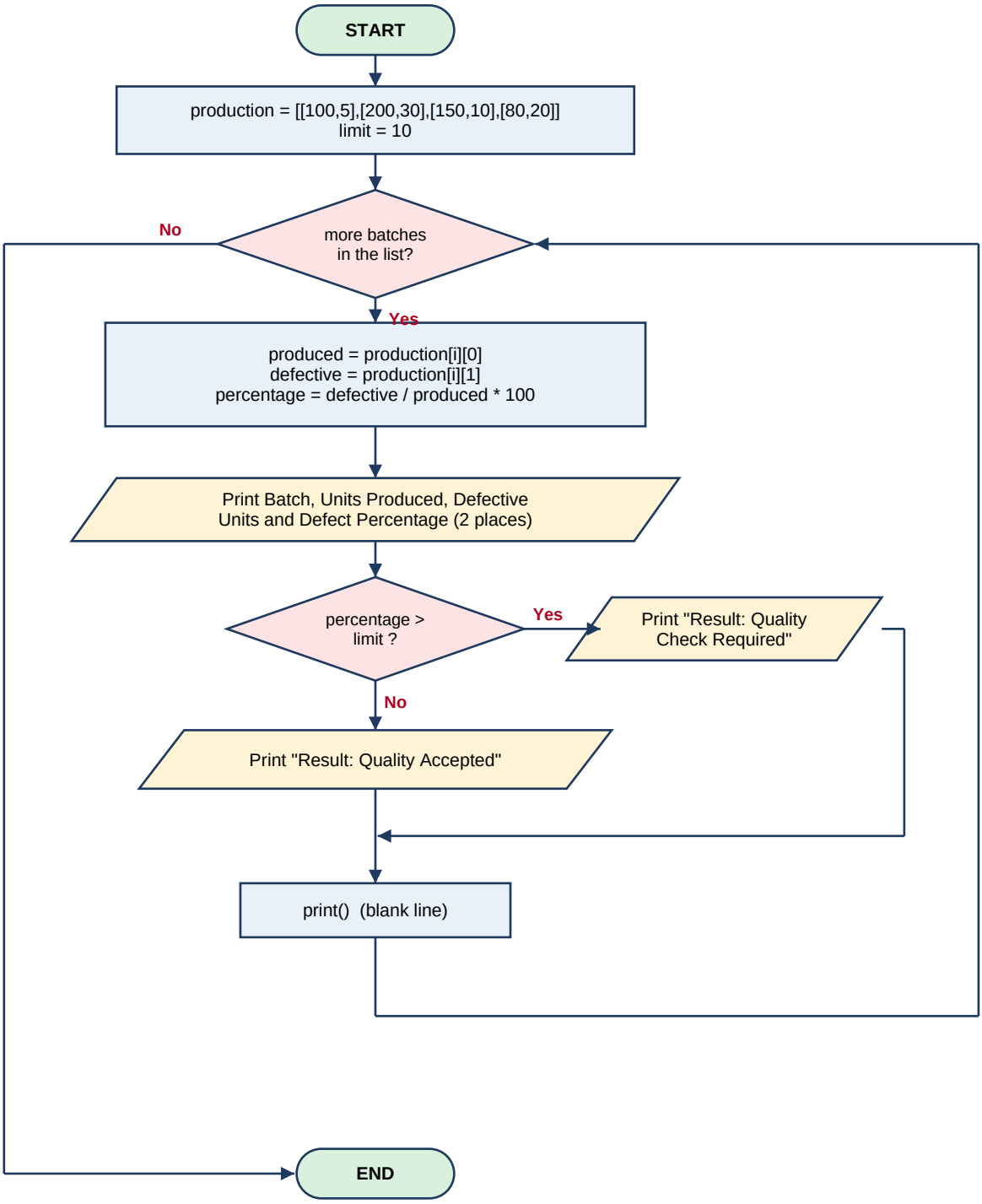
3. Assumptions

- Four batches are processed: [100, 5], [200, 30], [150, 10] and [80, 20], each as [units produced, defective units].
- Defect percentage = (defective units / units produced) x 100, rounded to 2 decimal places.
- The permitted defect limit is 10%; a batch needs a quality check only when its defect percentage is strictly greater than 10.
- Batches are numbered from 1 in the order they appear in the matrix.
- Units produced is never zero and defective units never exceed units produced.

4. Algorithm

1. Start.
2. Create the matrix production and set limit = 10.
3. For each batch index i from 0 to the number of batches - 1, repeat steps 4 to 8.
4. Read produced = production[i][0] and defective = production[i][1].
5. Calculate percentage = defective / produced x 100.
6. Display the batch number (i + 1), units produced, defective units and the defect percentage rounded to 2 decimals.
7. If percentage is greater than limit, display "Result: Quality Check Required"; otherwise display "Result: Quality Accepted".
8. Print a blank line and move to the next batch.
9. Stop.

5. Flowchart


START
production = [[100,5],[200,30],[150,10],[80,20]]
limit = 10
more batches in the list?
No
Yes
produced = production[i][0]
defective = production[i][1]
percentage = defective / produced * 100
Print Batch, Units Produced, Defective Units and Defect Percentage (2 places)
percentage > limit ?
Yes
Print "Result: Quality Check Required"
No
Print "Result: Quality Accepted"
print() (blank line)
END

6. Pseudocode

```
START
  SET production <- [[100,5], [200,30], [150,10], [80,20]]
  SET limit <- 10
  FOR i <- 0 TO LENGTH(production) - 1 DO
      produced <- production[i][0]
      defective <- production[i][1]
      percentage <- defective / produced * 100
      PRINT "Batch:", i + 1
      PRINT "Units Produced:", produced
      PRINT "Defective Units:", defective
      PRINT "Defect Percentage:", ROUND(percentage, 2), "%"
      IF percentage > limit THEN
          PRINT "Result: Quality Check Required"
      ELSE
          PRINT "Result: Quality Accepted"
      END IF
      PRINT blank line
  END FOR
END
```

7. Python Source Code

File name: **case7.pyt**

```
# Case Study 7: Manufacturing Quality Control

production = [
    [100, 5],
    [200, 30],
    [150, 10],
    [80, 20]
]

limit = 10

for i in range(len(production)):

    produced = production[i][0]
    defective = production[i][1]

    percentage = defective / produced * 100

    print("Batch:", i + 1)
    print("Units Produced:", produced)
    print("Defective Units:", defective)
    print("Defect Percentage:", round(percentage, 2), "%")

    if percentage > limit:
        print("Result: Quality Check Required")
    else:
        print("Result: Quality Accepted")

    print()
```

8. Test Cases

Test Case 7: Four production batches with a 10% defect limit; two batches are accepted and two require a quality check.

Item	Details
Test case ID	TC-7
Input (production matrix)	[[100, 5], [200, 30], [150, 10], [80, 20]], limit = 10
Conditions tested	Two-level indexing of a nested list, defect percentage calculation, accepted and quality-check branches
Expected output	Defect percentages 5.0, 15.0, 6.67, 25.0; Batches 2 and 4 require a quality check

Batch-wise calculation (expected result):

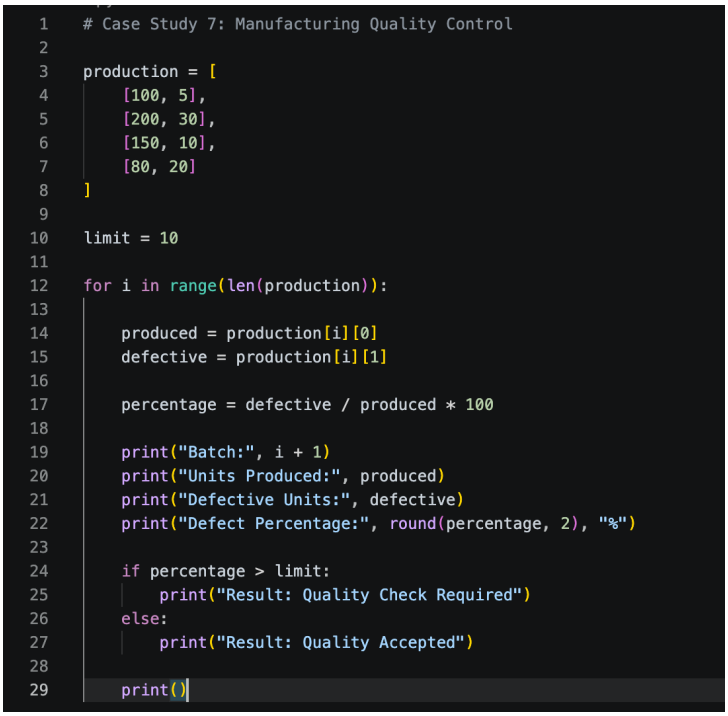
Batch	Produced	Defective	Percentage	Rounded	> 10 ?	Result
1	100	5	5/100 x 100	5.0	No	Quality Accepted
2	200	30	30/200 x 100	15.0	Yes	Quality Check Required
3	150	10	10/150 x 100	6.67	No	Quality Accepted
4	80	20	20/80 x 100	25.0	Yes	Quality Check Required

Check	Expected	Actual
Batches processed	4	4
Batches needing a quality check	2 (Batch 2 and Batch 4)	2 (Batch 2 and Batch 4)
Highest defect percentage	25.0 % (Batch 4)	25.0 % (Batch 4)

Expected output	Actual output	Result
Batch 4: Defect Percentage 25.0 %, Quality Check Required	Batch 4: Defect Percentage 25.0 %, Quality Check Required	PASS

9. Output Screenshots

Screenshot 1: Source code **case7.pyt** in VS Code.



```
# Case Study 7: Manufacturing Quality Control

production = [
    [100, 5],
    [200, 30],
    [150, 10],
    [80, 20]
]

limit = 10

for i in range(len(production)):

    produced = production[i][0]
    defective = production[i][1]

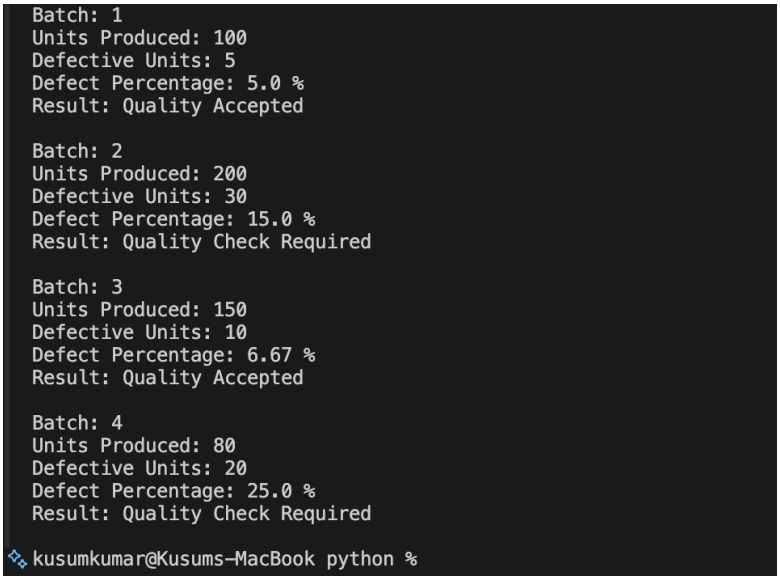
    percentage = defective / produced * 100

    print("Batch:", i + 1)
    print("Units Produced:", produced)
    print("Defective Units:", defective)
    print("Defect Percentage:", round(percentage, 2), "%")

    if percentage > limit:
        print("Result: Quality Check Required")
    else:
        print("Result: Quality Accepted")

    print()
```

Screenshot 2: Output of **case7.pyt** in the VS Code terminal (Python 3.14.7).



```
Batch: 1
Units Produced: 100
Defective Units: 5
Defect Percentage: 5.0 %
Result: Quality Accepted

Batch: 2
Units Produced: 200
Defective Units: 30
Defect Percentage: 15.0 %
Result: Quality Check Required

Batch: 3
Units Produced: 150
Defective Units: 10
Defect Percentage: 6.67 %
Result: Quality Accepted

Batch: 4
Units Produced: 80
Defective Units: 20
Defect Percentage: 25.0 %
Result: Quality Check Required

kusumkumar@Kusums-MacBook python %
```

10. Conclusion

The Manufacturing Quality Control Matrix program produced the correct report for Test Case 7. For the four batches it calculated defect percentages of 5.0, 15.0, 6.67 and 25.0. Batches 1 and 3 were accepted because they are within the 10% limit, while Batches 2 and 4 were correctly marked as requiring a quality check, matching the expected output. The program demonstrates nested lists with two-level indexing, a for loop with range() and len(), arithmetic operators, round() and if / else conditions.

Python Activity-Based Learning

Case Study 71: Integrated University Fee and Eligibility Audit

Test Case 8

1. Problem Statement

Develop a Python program that audits the fee payment and examination eligibility of university students. Each student record is stored as a tuple containing the roll number, fee paid, total fee, attendance percentage and internal marks. The program checks every eligibility condition, collects every unmet requirement of a student in a list, and displays whether the student is eligible or not eligible together with a reason-wise report.

2. Objectives

- To store student records as tuples inside a list and unpack each tuple into separate variables.
- To apply independent if conditions for fee payment, attendance and internal marks.
- To collect all unmet requirements of a student in a list using append().
- To use len() to decide eligibility (an empty reasons list means eligible).
- To generate a reason-wise audit report for every student.

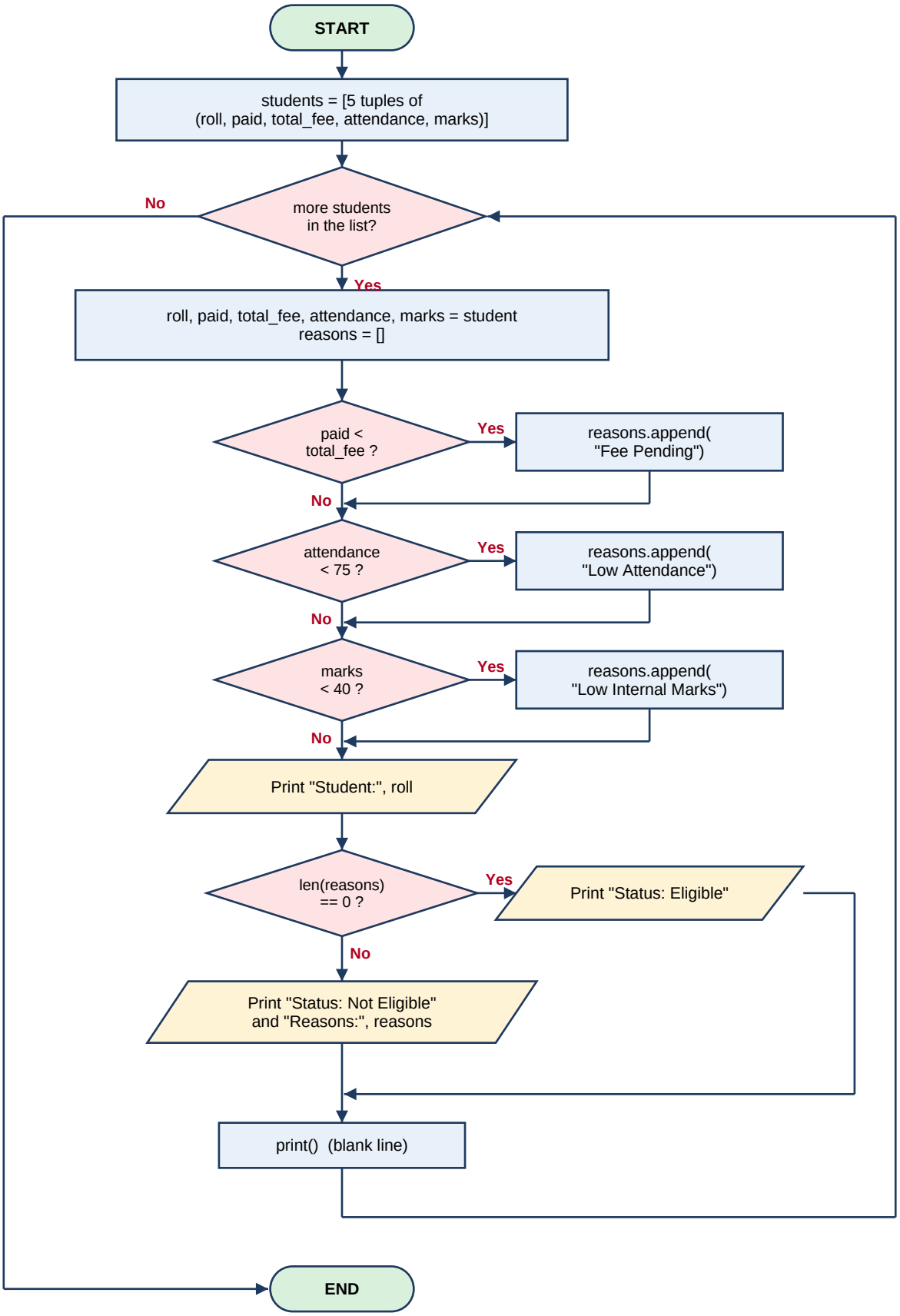
3. Assumptions

- Each record is a tuple (roll, paid, total_fee, attendance, marks); five students S101 to S105 are audited.
- Fee is cleared only when the fee paid is not less than the total fee; otherwise "Fee Pending" is reported.
- The minimum attendance required is 75%; a lower value is reported as "Low Attendance".
- The minimum internal marks required are 40; a lower value is reported as "Low Internal Marks".
- All three conditions are checked independently, so a student can have more than one reason.
- A student is eligible only when no requirement is unmet.

4. Algorithm

1. Start.
2. Create the list students of (roll, paid, total_fee, attendance, marks) tuples.
3. For each student, unpack the tuple into roll, paid, total_fee, attendance and marks; set reasons = [].
4. If paid is less than total_fee, add "Fee Pending" to reasons.
5. If attendance is less than 75, add "Low Attendance" to reasons.
6. If marks is less than 40, add "Low Internal Marks" to reasons.
7. Display the student's roll number.
8. If the reasons list is empty, display "Status: Eligible"; otherwise display "Status: Not Eligible" and the reasons.
9. Print a blank line and repeat steps 3 to 8 for the next student.
10. Stop.

5. Flowchart


START
students = [5 tuples of
(roll, paid, total_fee, attendance, marks)]
more students
in the list?
No
Yes
roll, paid, total_fee, attendance, marks = student
reasons = []
paid <
total_fee ?
Yes
reasons.append(
"Fee Pending")
No
attendance
< 75 ?
Yes
reasons.append(
"Low Attendance")
No
marks
< 40 ?
Yes
reasons.append(
"Low Internal Marks")
No
Print "Student:", roll
len(reasons)
== 0 ?
Yes
Print "Status: Eligible"
No
Print "Status: Not Eligible"
and "Reasons:", reasons
print() (blank line)
END

6. Pseudocode

```
START
  SET students <- list of tuples (roll, paid, total_fee, attendance, marks):
      ("S101",10000,10000,85,75), ("S102",8000,10000,90,80),
      ("S103",10000,10000,60,70), ("S104",10000,10000,80,30),
      ("S105",10000,10000,90,90)
  FOR EACH student IN students DO
      UNPACK student INTO roll, paid, total_fee, attendance, marks
      SET reasons <- empty list
      IF paid < total_fee THEN ADD "Fee Pending" TO reasons
      IF attendance < 75 THEN ADD "Low Attendance" TO reasons
      IF marks < 40 THEN ADD "Low Internal Marks" TO reasons
      PRINT "Student:", roll
      IF LENGTH(reasons) = 0 THEN
          PRINT "Status: Eligible"
      ELSE
          PRINT "Status: Not Eligible"
          PRINT "Reasons:", reasons
      END IF
      PRINT blank line
  END FOR
END
```

7. Python Source Code

File name: **case8.pyt**

```
# Case Study 8: University Fee and Eligibility Audit

students = [
    ("S101", 10000, 10000, 85, 75),
    ("S102", 8000, 10000, 90, 80),
    ("S103", 10000, 10000, 60, 70),
    ("S104", 10000, 10000, 80, 30),
    ("S105", 10000, 10000, 90, 90)
]

for student in students:

    roll, paid, total_fee, attendance, marks = student

    reasons = []

    if paid < total_fee:
        reasons.append("Fee Pending")

    if attendance < 75:
        reasons.append("Low Attendance")

    if marks < 40:
        reasons.append("Low Internal Marks")

    print("Student:", roll)

    if len(reasons) == 0:
        print("Status: Eligible")
    else:
        print("Status: Not Eligible")
        print("Reasons:", reasons)

    print()
```

8. Test Cases

Test Case 8: Five student records where two students satisfy all conditions and three students each fail exactly one condition.

Item	Details
Test case ID	TC-8
Input	Five tuples (roll, paid, total_fee, attendance, marks): S101 to S105 as given in the source code
Conditions tested	Tuple unpacking, independent fee / attendance / marks checks, reasons list and len() == 0 test
Expected output	S101 and S105 Eligible; S102 Fee Pending; S103 Low Attendance; S104 Low Internal Marks

Student-wise audit (expected result):

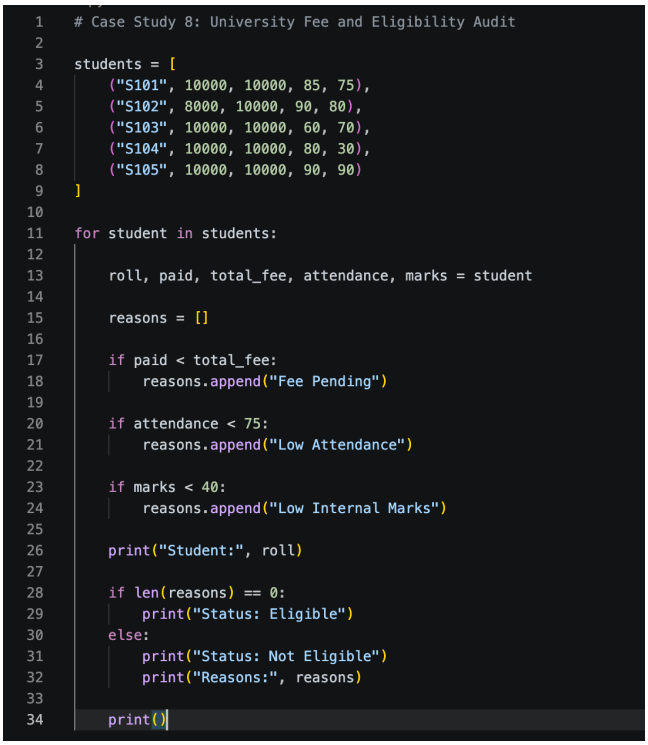
Roll	Paid / Total fee	Attendance	Marks	Reasons collected	Status
S101	10000 / 10000	85	75	none	Eligible
S102	8000 / 10000	90	80	Fee Pending (8000 < 10000)	Not Eligible
S103	10000 / 10000	60	70	Low Attendance (60 < 75)	Not Eligible
S104	10000 / 10000	80	30	Low Internal Marks (30 < 40)	Not Eligible
S105	10000 / 10000	90	90	none	Eligible

Check	Expected	Actual
Students audited	5	5
Eligible students	2 (S101, S105)	2 (S101, S105)
Not eligible students	3 (S102, S103, S104)	3 (S102, S103, S104)

Expected output	Actual output	Result
S101 Eligible; S102 [Fee Pending]; S103 [Low Attendance]; S104 [Low Internal Marks]; S105 Eligible	S101 Eligible; S102 [Fee Pending]; S103 [Low Attendance]; S104 [Low Internal Marks]; S105 Eligible	PASS

9. Output Screenshots

Screenshot 1: Source code **case8.pyt** in VS Code.



```
# Case Study 8: University Fee and Eligibility Audit

students = [
    ("S101", 10000, 10000, 85, 75),
    ("S102", 8000, 10000, 90, 80),
    ("S103", 10000, 10000, 60, 70),
    ("S104", 10000, 10000, 80, 30),
    ("S105", 10000, 10000, 90, 90)
]

for student in students:

    roll, paid, total_fee, attendance, marks = student

    reasons = []

    if paid < total_fee:
        reasons.append("Fee Pending")

    if attendance < 75:
        reasons.append("Low Attendance")

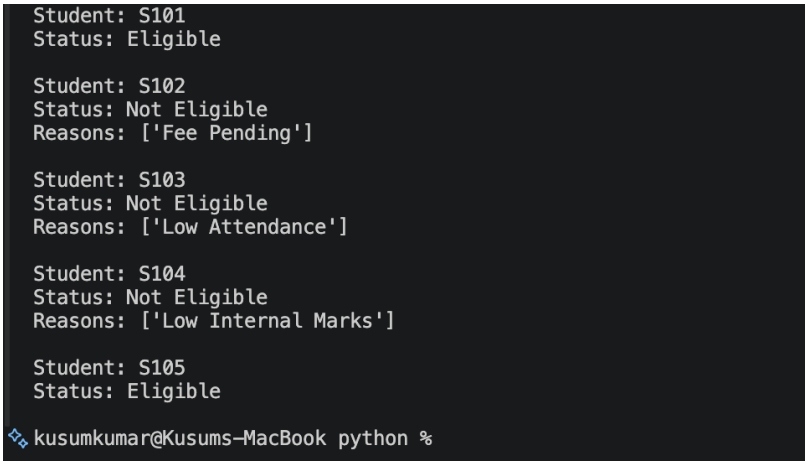
    if marks < 40:
        reasons.append("Low Internal Marks")

    print("Student:", roll)

    if len(reasons) == 0:
        print("Status: Eligible")
    else:
        print("Status: Not Eligible")
        print("Reasons:", reasons)

    print()
```

Screenshot 2: Output of **case8.pyt** in the VS Code terminal (Python 3.14.7).



```
Student: S101
Status: Eligible

Student: S102
Status: Not Eligible
Reasons: ['Fee Pending']

Student: S103
Status: Not Eligible
Reasons: ['Low Attendance']

Student: S104
Status: Not Eligible
Reasons: ['Low Internal Marks']

Student: S105
Status: Eligible

kusumkumar@Kusums-MacBook python %
```

10. Conclusion

The Integrated University Fee and Eligibility Audit produced the correct report for Test Case 8. Students S101 and S105 met every requirement and were reported as Eligible, while S102 (fee pending), S103 (low attendance) and S104 (low internal marks) were reported as Not Eligible with the exact reason listed, matching the expected output. The program demonstrates tuples with tuple unpacking, a list of reasons built with append(), independent if conditions, len() and if / else logic. Students failing more than one condition would list all of their reasons in the same way.

Python Activity-Based Learning

Case Study 74: Campus Festival Registration and Financial Audit

Test Case 9

1. Problem Statement

Develop a Python program to audit the registrations and finances of a campus festival. Each registration is stored as a tuple of registration ID and fee. The program must detect and reject duplicate registrations, count only the valid registrations, calculate the total registration fees, add the sponsorship amount to obtain the total income, subtract the expenses, and report whether the festival ends in a surplus, a deficit or break-even.

2. Objectives

- To store registrations as tuples (registration ID, fee) in a list and unpack each tuple.
- To detect duplicate registrations using the membership operator **in** on a list of registered IDs.
- To accumulate the registration fees of valid registrations using a running total.
- To calculate income and balance using arithmetic operators.
- To classify the final financial position as Surplus, Deficit or Break-even using if / elif / else.

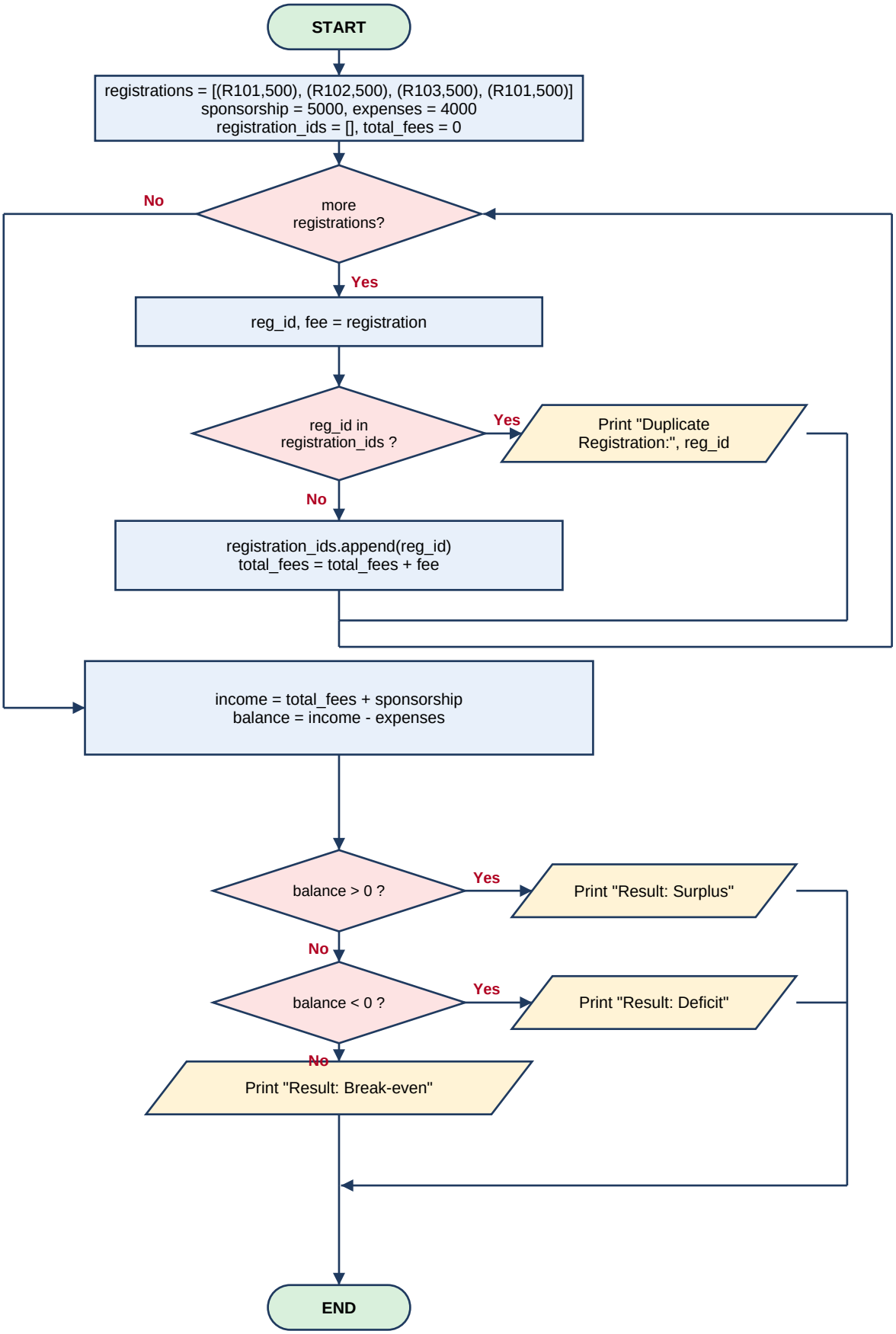
3. Assumptions

- Four registration records are given: (R101, 500), (R102, 500), (R103, 500) and (R101, 500), each as (registration ID, fee in rupees).
- A registration whose ID has already been registered is a duplicate; it is reported and its fee is not counted.
- The festival sponsorship is Rs. 5000 and the total expenses are Rs. 4000.
- Total income = total registration fees (valid registrations only) + sponsorship.
- Final balance = total income - total expenses; a positive balance is a surplus, a negative balance is a deficit and zero is break-even.

4. Algorithm

1. Start.
2. Create the list registrations, set sponsorship = 5000 and expenses = 4000, create an empty list registration_ids and set total_fees = 0.
3. For each registration, unpack the tuple into reg_id and fee.
4. If reg_id is already present in registration_ids, display "Duplicate Registration" with the ID.
5. Otherwise, add reg_id to registration_ids and add fee to total_fees.
6. Repeat steps 3 to 5 for every registration.
7. Calculate income = total_fees + sponsorship and balance = income - expenses.
8. Display the number of valid registrations, registration fees, sponsorship, total income, total expenses and final balance.
9. If balance > 0 display "Result: Surplus"; else if balance < 0 display "Result: Deficit"; otherwise display "Result: Break-even".
10. Stop.

5. Flowchart


START
registrations = [(R101,500), (R102,500), (R103,500), (R101,500)]
sponsorship = 5000, expenses = 4000
registration_ids = [], total_fees = 0
more registrations?
No
Yes
reg_id, fee = registration
reg_id in registration_ids ?
Yes
Print "Duplicate Registration:", reg_id
No
registration_ids.append(reg_id)
total_fees = total_fees + fee
income = total_fees + sponsorship
balance = income - expenses
balance > 0 ?
Yes
Print "Result: Surplus"
No
balance < 0 ?
Yes
Print "Result: Deficit"
No
Print "Result: Break-even"
END

6. Pseudocode

```
START
  SET registrations <- [("R101",500), ("R102",500), ("R103",500), ("R101",500)]
  SET sponsorship <- 5000
  SET expenses <- 4000
  SET registration_ids <- empty list
  SET total_fees <- 0
  FOR EACH registration IN registrations DO
      UNPACK registration INTO reg_id, fee
      IF reg_id IS IN registration_ids THEN
          PRINT "Duplicate Registration:", reg_id
      ELSE
          ADD reg_id TO registration_ids
          total_fees <- total_fees + fee
      END IF
  END FOR
  income <- total_fees + sponsorship
  balance <- income - expenses
  PRINT valid registrations (LENGTH of registration_ids), total_fees,
        sponsorship, income, expenses and balance
  IF balance > 0 THEN PRINT "Result: Surplus"
  ELSE IF balance < 0 THEN PRINT "Result: Deficit"
  ELSE PRINT "Result: Break-even"
  END IF
END
```

7. Python Source Code

File name: **case9.pyt**

```
# Case Study 9: Campus Festival Financial Audit

registrations = [
    ("R101", 500),
    ("R102", 500),
    ("R103", 500),
    ("R101", 500)
]

sponsorship = 5000
expenses = 4000

registration_ids = []
total_fees = 0

for registration in registrations:

    reg_id, fee = registration

    if reg_id in registration_ids:
        print("Duplicate Registration:", reg_id)
    else:
        registration_ids.append(reg_id)
        total_fees = total_fees + fee

income = total_fees + sponsorship
balance = income - expenses

print("Valid Registrations:", len(registration_ids))
print("Registration Fees:", total_fees)
print("Sponsorship:", sponsorship)
print("Total Income:", income)
print("Total Expenses:", expenses)
print("Final Balance:", balance)

if balance > 0:
    print("Result: Surplus")
elif balance < 0:
    print("Result: Deficit")
else:
    print("Result: Break-even")
```

8. Test Cases

Test Case 9: Four registrations of which one (R101) is a duplicate; income exceeds expenses, giving a surplus.

Item	Details
Test case ID	TC-9
Input	registrations = (R101, 500), (R102, 500), (R103, 500), (R101, 500); sponsorship = 5000; expenses = 4000
Conditions tested	Tuple unpacking, duplicate detection with **in**, running total, income and balance calculation, surplus branch
Expected output	Duplicate Registration: R101; Valid Registrations: 3; Registration Fees: 1500; Total Income: 6500; Final Balance: 2500; Result: Surplus

Registration-wise trace (expected result):

#	Registration	Fee	Already in registration_ids?	Action	total_fees
1	R101	500	No	Add R101	500
2	R102	500	No	Add R102	1000
3	R103	500	No	Add R103	1500
4	R101	500	Yes	Duplicate - rejected	1500

Calculation	Evaluation	Result
Valid registrations	len([R101, R102, R103])	3
Total income	1500 + 5000	6500
Final balance	6500 - 4000	2500
Classification	balance > 0	Surplus

Expected output	Actual output	Result
Duplicate Registration: R101, Valid Registrations: 3, Final Balance: 2500, Result: Surplus	Duplicate Registration: R101, Valid Registrations: 3, Final Balance: 2500, Result: Surplus	PASS

9. Output Screenshots

Screenshot 1: Source code **case9.pyt** in VS Code.



```python
# Case Study 9: Campus Festival Financial Audit

registrations = [
    ("R101", 500),
    ("R102", 500),
    ("R103", 500),
    ("R101", 500)
]

sponsorship = 5000
expenses = 4000

registration_ids = []
total_fees = 0

for registration in registrations:

    reg_id, fee = registration

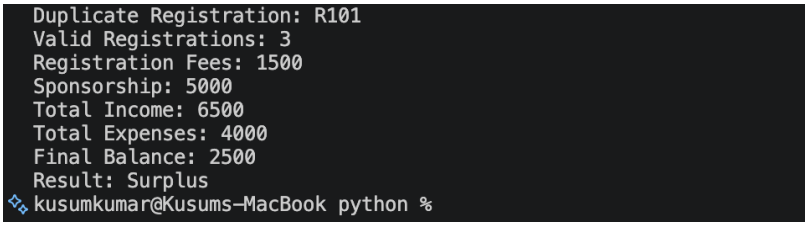
    if reg_id in registration_ids:
        print("Duplicate Registration:", reg_id)
    else:
        registration_ids.append(reg_id)
        total_fees = total_fees + fee

income = total_fees + sponsorship
balance = income - expenses

print("Valid Registrations:", len(registration_ids))
print("Registration Fees:", total_fees)
print("Sponsorship:", sponsorship)
print("Total Income:", income)
print("Total Expenses:", expenses)
print("Final Balance:", balance)

if balance > 0:
    print("Result: Surplus")
elif balance < 0:
    print("Result: Deficit")
else:
    print("Result: Break-even")
```

Screenshot 2: Output of **case9.pyt** in the VS Code terminal (Python 3.14.7).



```
Duplicate Registration: R101
Valid Registrations: 3
Registration Fees: 1500
Sponsorship: 5000
Total Income: 6500
Total Expenses: 4000
Final Balance: 2500
Result: Surplus
kusumkumar@Kusums-MacBook python %
```

10. Conclusion

The Campus Festival Registration and Financial Audit produced the correct report for Test Case 9. The duplicate registration R101 was detected and rejected, leaving 3 valid registrations and registration fees of Rs. 1500. Adding the Rs. 5000 sponsorship gave a total income of Rs. 6500, and after subtracting the expenses of Rs. 4000 the final balance was Rs. 2500, correctly classified as a Surplus. The program demonstrates tuples with unpacking, list membership testing with **in**, append(), a running total, arithmetic operators and if / elif / else conditions. The Deficit and Break-even branches are covered by further test cases.

Python Activity-Based Learning

Case Study 87: Examination Question Difficulty and Performance Analysis

Test Case 10

1. Problem Statement

Develop a Python program that analyses the answers given by students in an examination. Each student's answers are stored as a list and compared, question by question, with the answer key. The program calculates the number of correct and incorrect answers of every student, counts how many students answered each question correctly, calculates the success rate of every question, and classifies each question as Difficult, Moderate or Easy.

2. Objectives

- To store an answer key and the students' answers as a list and a list of lists.
- To use nested **for** loops with range() and two-level indexing (students[i][j]) to compare answers with the key.
- To keep per-student scores and per-question counts using variables and a counter list.
- To calculate question-wise success rates using arithmetic operators and round().
- To classify question difficulty using if / elif / else conditions.

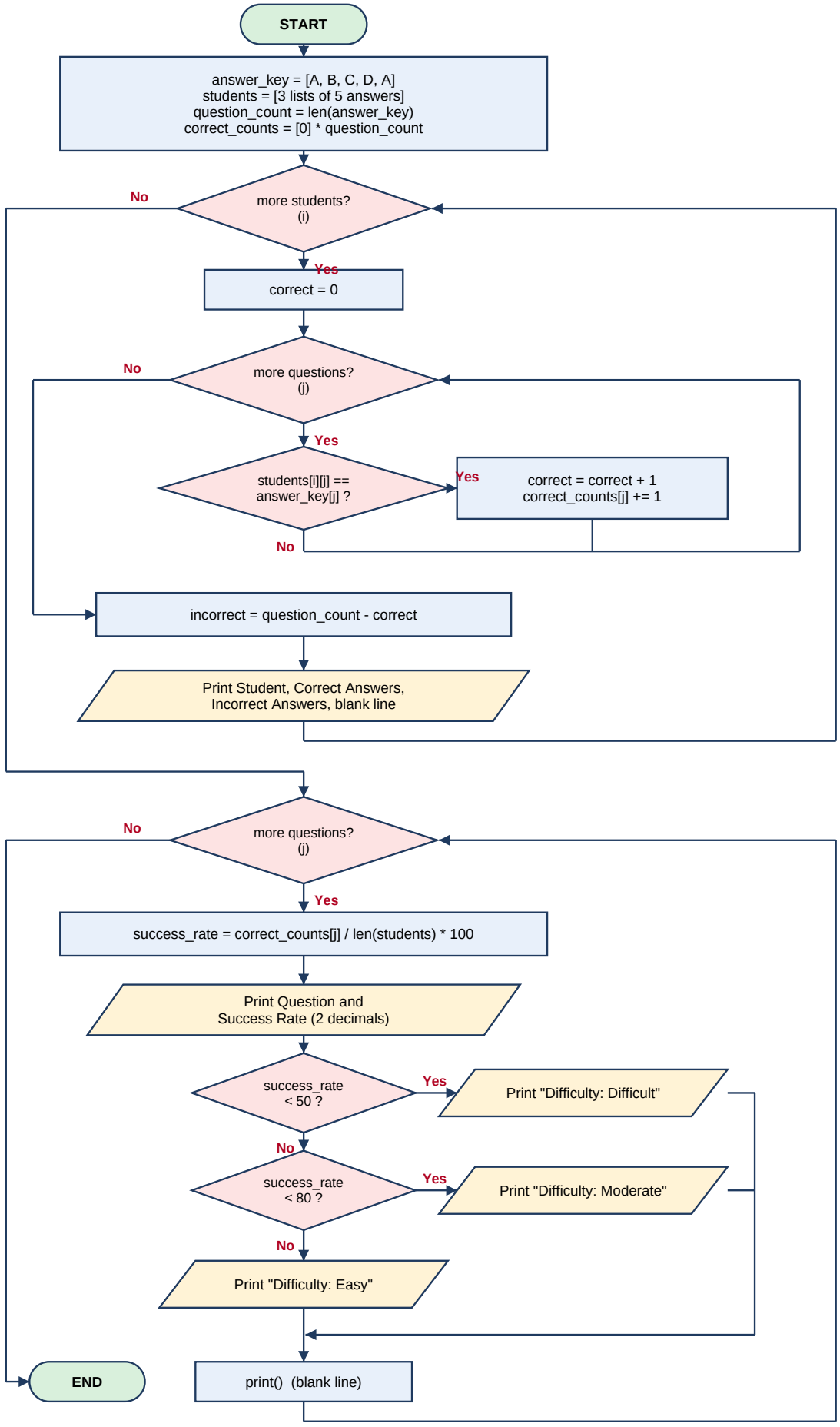
3. Assumptions

- The answer key is [A, B, C, D, A] (5 questions) and 3 students have answered all 5 questions.
- Each student's answers are stored in question order, so students[i][j] is student i's answer to question j.
- Success rate of a question = (number of students who answered it correctly / total students) x 100, rounded to 2 decimals.
- Difficulty classification: success rate below 50 - Difficult; 50 to below 80 - Moderate; 80 and above - Easy.
- Students and questions are numbered from 1 in the output.

4. Algorithm

1. Start.
2. Create answer_key and the students list; set question_count = len(answer_key) and correct_counts = [0] * question_count.
3. For each student i, set correct = 0.
4. For each question j, if students[i][j] equals answer_key[j], add 1 to correct and add 1 to correct_counts[j].
5. Calculate incorrect = question_count - correct and display the student number, correct and incorrect answers.
6. Repeat steps 3 to 5 for every student.
7. For each question j, calculate success_rate = correct_counts[j] / number of students x 100.
8. Display the question number and the success rate rounded to 2 decimals.
9. If success_rate < 50 display "Difficult"; else if success_rate < 80 display "Moderate"; otherwise display "Easy".
10. Repeat steps 7 to 9 for every question.
11. Stop.

5. Flowchart


START
answer_key = [A, B, C, D, A]
students = [3 lists of 5 answers]
question_count = len(answer_key)
correct_counts = [0] * question_count
more students?
(i)
No
Yes
correct = 0
more questions?
(j)
No
Yes
students[i][j] ==
answer_key[j] ?
Yes
correct = correct + 1
correct_counts[j] += 1
No
incorrect = question_count - correct
Print Student, Correct Answers,
Incorrect Answers, blank line
more questions?
(j)
No
Yes
success_rate = correct_counts[j] / len(students) * 100
Print Question and
Success Rate (2 decimals)
success_rate
< 50 ?
Yes
Print "Difficulty: Difficult"
No
success_rate
< 80 ?
Yes
Print "Difficulty: Moderate"
No
Print "Difficulty: Easy"
END
print() (blank line)

6. Pseudocode

```
START
  SET answer_key <- ["A","B","C","D","A"]
  SET students <- [["A","B","C","D","A"], ["A","C","C","D","B"], ["B","B","C","A","A"]]
  SET question_count <- LENGTH(answer_key)
  SET correct_counts <- list of question_count zeros
  FOR i <- 0 TO LENGTH(students) - 1 DO
      SET correct <- 0
      FOR j <- 0 TO question_count - 1 DO
          IF students[i][j] = answer_key[j] THEN
              correct <- correct + 1
              correct_counts[j] <- correct_counts[j] + 1
          END IF
      END FOR
      incorrect <- question_count - correct
      PRINT "Student:", i + 1, correct, incorrect, blank line
  END FOR
  FOR j <- 0 TO question_count - 1 DO
      success_rate <- correct_counts[j] / LENGTH(students) * 100
      PRINT "Question:", j + 1, ROUND(success_rate, 2)
      IF success_rate < 50 THEN PRINT "Difficulty: Difficult"
      ELSE IF success_rate < 80 THEN PRINT "Difficulty: Moderate"
      ELSE PRINT "Difficulty: Easy"
      END IF
      PRINT blank line
  END FOR
END
```

7. Python Source Code

File name: **case10.pyt**

```
 1  # Case Study 10: Examination Question Analysis
 2  answer_key = ["A", "B", "C", "D", "A"]
 3  students = [
 4      ["A", "B", "C", "D", "A"],
 5      ["A", "C", "C", "D", "B"],
 6      ["B", "B", "C", "A", "A"]
 7  ]
 8  question_count = len(answer_key)
 9  correct_counts = [0] * question_count
10  for i in range(len(students)):
11      correct = 0
12      for j in range(question_count):
13
14          if students[i][j] == answer_key[j]:
15              correct = correct + 1
16              correct_counts[j] = correct_counts[j] + 1
17      incorrect = question_count - correct
18
19      print("Student:", i + 1)
20      print("Correct Answers:", correct)
21      print("Incorrect Answers:", incorrect)
22      print()
23  for j in range(question_count):
24      success_rate = correct_counts[j] / len(students) * 100
25      print("Question:", j + 1)
26      print("Success Rate:", round(success_rate, 2), "%")
27      if success_rate < 50:
28          print("Difficulty: Difficult")
29      elif success_rate < 80:
30          print("Difficulty: Moderate")
31      else:
32          print("Difficulty: Easy")
33      print()
```

8. Test Cases

Test Case 10: Three students answer a five-question paper; the answers of each student are compared with the answer key.

Item	Details
Test case ID	TC-10
Input	answer_key = [A, B, C, D, A]; Student 1 = [A, B, C, D, A]; Student 2 = [A, C, C, D, B]; Student 3 = [B, B, C, A, A]
Conditions tested	Nested loops, comparison students[i][j] == answer_key[j], per-question counter list, success rate, Easy and Moderate branches
Expected output	Student scores 5, 3, 3; success rates 66.67, 66.67, 100.0, 66.67, 66.67; Question 3 Easy, all others Moderate

Student-wise evaluation (expected result):

Student	Answers	Wrong questions	Correct	Incorrect
1	A B C D A	none	5	0
2	A C C D B	Q2 (C vs B), Q5 (B vs A)	3	2
3	B B C A A	Q1 (B vs A), Q4 (A vs D)	3	2

Question-wise analysis (expected result):

Question	Key	Correct by	Count	Success rate	< 50 ?	< 80 ?	Difficulty
1	A	Student 1, 2	2	2/3 x 100 = 66.67	No	Yes	Moderate
2	B	Student 1, 3	2	66.67	No	Yes	Moderate
3	C	Student 1, 2, 3	3	3/3 x 100 = 100.0	No	No	Easy
4	D	Student 1, 2	2	66.67	No	Yes	Moderate
5	A	Student 1, 3	2	66.67	No	Yes	Moderate

Expected output	Actual output	Result
Student 1: 5 / 0, Student 2: 3 / 2, Student 3: 3 / 2; Q3 100.0 % Easy; other questions 66.67 % Moderate	Student 1: 5 / 0, Student 2: 3 / 2, Student 3: 3 / 2; Q3 100.0 % Easy; other questions 66.67 % Moderate	PASS

9. Output Screenshots

Screenshot 1: Source code **case10.pyt** in VS Code.

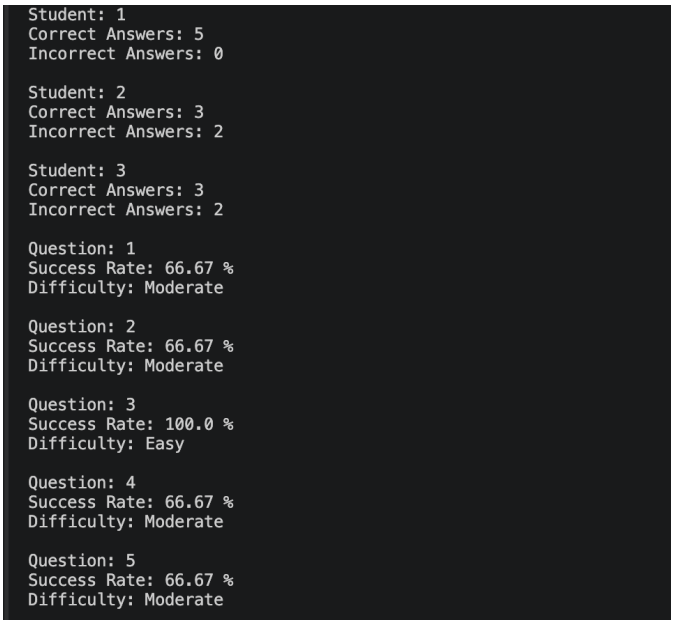


```
# Case Study 10: Examination Question Analysis
answer_key = ["A", "B", "C", "D", "A"]
students = [
    ["A", "B", "C", "D", "A"],
    ["A", "C", "C", "D", "B"],
    ["B", "B", "C", "A", "A"]
]
question_count = len(answer_key)
correct_counts = [0] * question_count
for i in range(len(students)):
    correct = 0
    for j in range(question_count):

        if students[i][j] == answer_key[j]:
            correct = correct + 1
            correct_counts[j] = correct_counts[j] + 1
    incorrect = question_count - correct

    print("Student:", i + 1)
    print("Correct Answers:", correct)
    print("Incorrect Answers:", incorrect)
    print()
for j in range(question_count):
    success_rate = correct_counts[j] / len(students) * 100
    print("Question:", j + 1)
    print("Success Rate:", round(success_rate, 2), "%")
    if success_rate < 50:
        print("Difficulty: Difficult")
    elif success_rate < 80:
        print("Difficulty: Moderate")
    else:
        print("Difficulty: Easy")
    print()
```

Screenshot 2: Output of **case10.pyt** in the VS Code terminal (Python 3.14.7).



```
Student: 1
Correct Answers: 5
Incorrect Answers: 0

Student: 2
Correct Answers: 3
Incorrect Answers: 2

Student: 3
Correct Answers: 3
Incorrect Answers: 2

Question: 1
Success Rate: 66.67 %
Difficulty: Moderate

Question: 2
Success Rate: 66.67 %
Difficulty: Moderate

Question: 3
Success Rate: 100.0 %
Difficulty: Easy

Question: 4
Success Rate: 66.67 %
Difficulty: Moderate

Question: 5
Success Rate: 66.67 %
Difficulty: Moderate
```

10. Conclusion

The Examination Question Difficulty and Performance Analysis program produced the correct results for Test Case 10. Student 1 answered all 5 questions correctly, while Students 2 and 3 answered 3 correctly and 2 incorrectly. Question 3 was answered correctly by all three students (100.0%) and was classified as Easy, while every other question had a success rate of 66.67% and was classified as Moderate, matching the expected output. The program demonstrates lists of lists, nested for loops with two-level indexing, a counter list, arithmetic operators, round() and if / elif / else conditions. The Difficult branch (below 50%) is covered by further test cases.